

Things to discuss in class: $\rightarrow$ Why automata? $\rightarrow$ When automata?
 $\rightarrow$ Where automata? $\rightarrow$ What is an automata?
 $\rightarrow$ "Automate" 3

Date: _____

ALPHABET & WORDS

Σ = Set of symbols | Alphabet | letters of alphabet
 Λ or λ or ϵ = ^{Null String} Empty word | word containing no letters
 Φ = Empty Set | Set containing no words.
 Σ^* = ~~subset~~ set of all words over Σ , including Λ
 $|x|$ = length of a word "x"

Concatenation:

$$\underline{PQ} = \{x \mid x = pq, p \text{ is in } P \text{ and } q \text{ is in } Q\}$$

Subsets of Σ^* $\leftarrow$

$$PQ \neq QP \rightarrow$$

$$(PQ)R = P(QR) \rightarrow$$

Property

Property

Example:

$$\Sigma = \{a, b\}, P = \{aa, bbb\}, Q = \{aba, bab\}$$

$$PQ = \{aaaba, aabab, bbbaba, bbbbab\}$$

$$QP = \{abaaa, \text{babaa}, ababbb, babbbb\}$$

Assume $S = \{0, 1\}$

$$S^0, S^1, S^2$$

$$\wedge \quad \{0, 1\} \quad \{00, 01, 10, 11\}$$

If $S = \{01, 11\}$

$$S^0 = \Lambda, S^1 = \{01, 11\}, S^2 = \{0101, 0111, 1101, 1111\}$$

$\rightarrow$ max number of non-empty characters? Cardinality.

→ Regular Sets :

Date: _____

A set of words over Σ that is recognized by a FSM is regular & regular set can be recognized by some FSM.

- (i) → every finite set of words over Σ (including $\emptyset$) is regular.
- (ii) → If P & Q are regular, the $P \cup Q$ & PQ are regular.
- (iii) → If S is a regular set over Σ , then so is S^* (closure of S)
- (iv) → No set is regular unless it's obtained by a finite no. of applications of (i) to (iii)

regular set over Σ is the smallest class containing all finite sets of words over Σ & closed under union, concatenation and star operation.

S^+ = all words of S^* except Λ (null string)

$$x^+ = x^* - \Lambda$$

If $\Rightarrow x = a$ Then, $x^+ = \{a, aa, aaa, \dots\}$

Every finite set is regular but not every regular set is finite
example $\rightarrow$ even numbers

Regular Expression : (RE)

Symbols that can appear in a RE :

• letters of alphabet " Σ " • Λ (null string)

• parenthesis • $*$ operator • $+$ sign in a RE means union

→ Regular Languages

• if a regex is just one letter, the language is that one letter

• if a regex r_1 is for language L_1 & r_2 for L_2 then :

language $(r_1 r_2) = L_1 L_2$ Product

language $(r_1 + r_2) = L_1 + L_2$ union

language $(r_1^*) = L_1^*$ Kleen closure of the set L_1

All finite Languages are regular

Suppose $\Rightarrow L = \{baa, abba, bababa\}$

the regex would be: $baa + abba + bababa$

Suppose $\Rightarrow L = \{aa, ab, ba, bb\}$

regex: $aa + ab + ba + bb$

we can simplify it:

$$a(a+b) + b(a+b)$$

$$(a+b)(a+b) \text{ // still a regex}$$

$$(a+b)^2 \text{ // not a regex. '2' is not a part of regex.}$$

Suppose $\Rightarrow L = \{\Lambda, x, xx, xxx, xxxx, xxxxx, xxxxxx\}$

regex = $\Lambda + x + xx + xxx + xxxx + xxxxx + xxxxxx$

How is this

$$\leftarrow x(\Lambda + x) + xx(\Lambda + x) + xxx(\Lambda + x)$$

simplified to $(\Lambda + x)^5$

$$(\Lambda + x)^3 (x(\Lambda + x) + xx(\Lambda + x))$$

$$x(\Lambda + x) + xx(\Lambda + x) + xxx(\Lambda + x)$$

Examples:

$$L = (a^*(a+b))^* = L(a^*)L(a+b)$$

$$= (L(a))^* (L(a) \cup L(b))$$

SET THEORY

Date: _____

$$\text{Set } S = \{1, 2, 3\}.$$

$\in$ $\Rightarrow$ Belongs

$$1 \in S$$

$$2 \in S$$

$$4 \notin S$$

$\rightarrow$ SET BUILDER NOTATION: If we don't want to write every element like above example.

$$P = \{ p \mid p \text{ is a prime} \}$$

$\xrightarrow{\text{var}} \quad \quad \quad \xrightarrow{\text{predicate}}$

$$\{ p \in \mathbb{N} \mid p < 5 \} \neq \{ r \in \mathbb{R} \mid r < 5 \}$$

$\xrightarrow{\text{natural number}} \quad \quad \quad \xrightarrow{\text{real numbers}}$

$\rightarrow$ EQUALITY:

$$B = \{1, 2, 3\} = A = \{3, 2, 1\}$$

order doesn't matter

$\rightarrow$ CARDINALITY:

$$A = \{1, 2, 3\} \text{ , then } |A| = 3$$
$$P = \{p \mid p \text{ is prime}\} \text{ , then } |P| = \infty$$

$\hookrightarrow$ cardinality.

$\rightarrow$ SUBSET:

$$A = \{2, 4, 6\} \text{ , } B = \{1, 2, 3, 4, 5, 6\}$$

then $\rightarrow A \subseteq B$

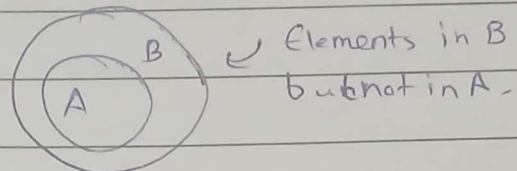
$\hookrightarrow$ sub set of

Is 4 a subset of B? or is $\{4\}$ a subset of B?

If $A \subseteq B$ & $B \subseteq A$, then $B = A$

PROPER SUBSET:

If $A \subseteq B$ & $A \neq B$, then A is a proper subset of B .



$A \subset B$ ← proper subset

Empty set $\rightarrow \emptyset \rightarrow$ is a subset of any set.

$\emptyset \rightarrow$ is unique.

Union & Intersection

$$A \cup B = \{x \mid x \in A \text{ (or) } x \in B\}$$

$$A \cap B = \{x \mid x \in A \text{ (and) } x \in B\}$$

Some Identities:

$$A \cup \emptyset = A$$

$$A \cup A = A$$

$$\text{If } A \subseteq B, \text{ then } A \cup B = B.$$

$$A \cup (B \cap C) = (A \cup B) \cap C$$

$$A \cap \emptyset = \emptyset$$

$$A \cap A = A$$

$$\text{If } A \subseteq B, \text{ then } A \cap B = A$$

$$A \cap (B \cap C) = (A \cap B) \cap C.$$

Date: _____

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

$$A - B \Rightarrow A = \{1, 2, 3, 4, 5\} \text{ \& } B = \{2, 4, 6, 8\}$$

$$B - A = \{6, 8\}$$

R is a relation on
(a, b)

Relation $\Downarrow$
 $\hookrightarrow aRb$
 from domain $\hookrightarrow$ range

- ⑤ Asymmetric

Example (Do in class) $\rightarrow S = \{ab, ba\}$

① Write S^* upto max 7 characters.

② Are there any substrings aaa, bbb ?

KLEENE ($*$) OR KLEENE CLOSURE

Date: _____

All possible combination of the alphabet including λ

$$S = \{a\}$$

$$\text{then } S^* = \{\lambda, a, aa, aaa, \dots\} \quad \left\| \begin{array}{l} \text{important!} \end{array} \right.$$

$$\text{If } S = \{a, b\}$$

$$\text{then } S^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, baab, \dots\}$$

Extended Kleene ($+$) $\rightarrow$ Kleene ($*$) but without $\lambda \rightarrow$ Positive Closure

STRINGS & LANGUAGES.

Alphabet $\rightarrow$ set of symbols/characters

Eg: English Alphabet / Binary Alphabet

Strings: $\rightarrow$ Made with symbols/characters from the alphabet.
(dog / cat $\in$ is a string over English Alphabet)

Length of a String: $|x|$ (given x is a string)

Empty String: A string with 0 length. λ or ϵ (lambda/Epsilon)

Reverse String: if $w = w_1 w_2 w_3 \dots w_n$ then --- reverse of w is $w_n w_{n-1} w_{n-2} \dots w_1$

Substring: If a string appears consecutively in another string, then it's a substring of the other string.

Cat $\rightarrow$ Catastrophic

Concatenation: If x & y are two strings, then if we append y to x , they are a concatenation $\Rightarrow xy$
 $x = \text{can}$ $y = \text{not}$
 $xy = \text{cannot}$

You can add a superscript when there's multiple strings of the same
 $xxx \Leftrightarrow x^3 \Rightarrow \text{cancancan}$

Suffix If $w = xv$, then v is a suffix of x .

Prefix If $w = vy$, then v is a prefix of y .

Lexicographic Ordering: Sorting/Ordering of strings similar to how they would be in a dictionary.

& Short strings come before long ones.

LANGUAGE: A subset of Σ^* for some alphabet Σ .
(Kleene Star mentioned earlier).

Alphabet Set $\Rightarrow \Sigma = \{x\}$ Kleene Closure

Kleene Star $\rightarrow \Sigma^* = \{ \lambda, x, xx, xxx, \dots \}$

Language $\rightarrow L_1 = \{x, xxx, \dots\}$

$L_2 = \{x, xx, xxx, \dots\}$

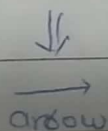
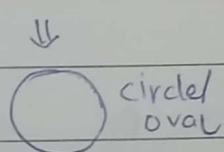
! can be many!

$L_3 = \{x, xx, xxx, \dots\}$

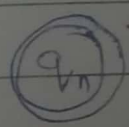
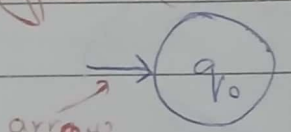
Can be written as: $L_3 = \{x^n \text{ for } n = 1, 2, 3, \dots\}$

AUTOMATA

States & Transitions



Types $\rightarrow$ (of states)



$\rightarrow$ there can be multiple final states

$(a^*b^*)^*$

$(a^*b^*)^*$

Date: _____

languages

is $a^*b^* = (ab)^*$?
a a a c b a b

$\{ +, \cdot, * \}$

$(a^* + c^*)b^*$

+

$(a, aa, aaa)b^* \mid (c, cc, ccc)b^*$

language such that it doesn't contain double letters.

$(\Lambda + b)(ab)^*(\Lambda + a)$

Set of strings of a's & b's not containing "bab"

doesn't contain aba.

a^*b^*b

$(a^*b^*(a^*a)^*a^*b^*)^*$
 $(aba)^* (ba)^*$
abaaba baba?

$(a^*b^*(a^*a)^*a^*b^*)^*$

Not containing 101

$(ba)^*$

$0^*(1^*00^*)^*1^*0^*$

baba

1, 1, 1

$a^*b^*(a^* + a)^*$

$(a^* + a)^*$

a^*

a^*

Date: _____

Even-Even. even a's & b's. → aabb bb aa

abab ba ba

aa bb

ab aa ab

ab bb ab

even number of a's

$$b^* + (b^* a b^* a b^*)^*$$

2-b
2-bb
2-bb

aa-bb

aaaa-bbb

babab

abab

bbaa

$$a^* + (a^* b a^* b a^*)^*$$

abab
abba
baab

What is an FS automata

((finite state))

FA or FSM $\Rightarrow (Q, \Sigma, \delta, q_0, F)$

Q = Set of States.

Σ = alphabet set

δ = Transition $\Rightarrow \delta(q_1, a) = q_2$

q_0 = start state $q_0 \in Q$

F = set of final states $F \subseteq Q$

$z(Q \times \Sigma) \rightarrow Q$
 complete string one state

Power Set of $q_0, q_1, q_2, q_3, q_4, q_5$

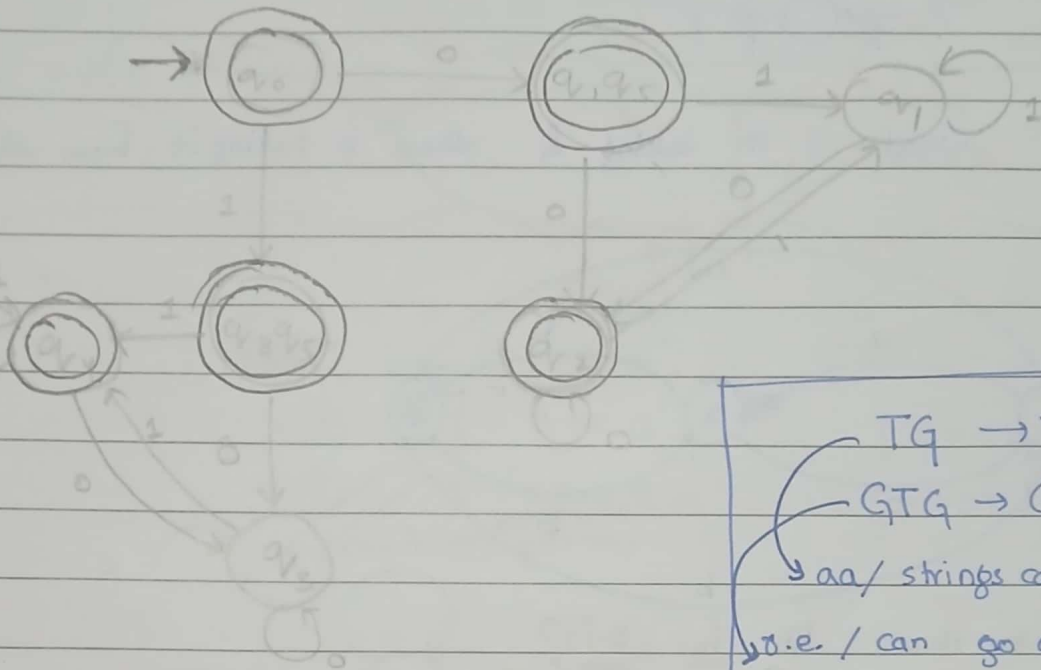
Date: _____

ϕ	ϕ	ϕ	q_1, q_2, q_3	q_2, q_3	q_1, q_4
✓ q_0	q_1, q_5	q_3, q_5	q_1, q_2, q_4	q_2, q_3	q_1, q_4
✓ q_1	q_2	q_1	q_1, q_2, q_5	q_2	q_1
✓ q_2	q_2	q_1	q_1, q_3, q_4	q_2, q_3	q_1, q_4
✓ q_3	q_3	q_4	q_2, q_3, q_5	q_2, q_3	q_2, q_3
✓ q_4	q_3	q_4	q_3, q_4, q_5	q_3	q_4
q_5	ϕ	ϕ	q_0, q_1, q_2, q_3	q_1, q_2, q_3, q_5	q_1, q_3, q_4, q_5
q_0, q_1	q_1, q_2, q_5	q_1, q_3, q_5	q_0, q_1, q_2, q_4	q_1, q_2, q_3, q_5	q_1, q_2, q_3, q_5
q_0, q_2	q_1, q_2, q_5	q_1, q_3, q_5	q_0, q_1, q_2, q_5	q_1, q_2, q_5	q_1, q_3, q_5
q_0, q_3	q_1, q_3, q_5	q_3, q_4, q_5	q_1, q_2, q_3, q_4	q_2, q_3	q_1, q_4
q_0, q_4	q_1, q_3, q_5	q_3, q_4, q_5	q_1, q_2, q_3, q_4, q_5	q_2, q_3	q_2, q_3
q_0, q_5	q_1, q_5	q_3, q_5	q_2, q_3, q_4, q_5	q_2, q_3	q_2, q_3
q_1, q_2	q_2	q_1	q_1, q_2, q_3, q_4, q_5	q_1, q_2, q_3, q_5	q_1, q_3, q_4, q_5
q_2, q_3	q_2, q_3	q_1, q_4	q_1, q_3, q_4	q_2, q_3	q_1, q_4
q_1, q_4	q_2, q_3	q_1, q_4	q_1, q_3, q_5	q_2, q_3	q_2, q_3
✓ q_1, q_5	q_2	q_1	q_1, q_4, q_5	q_2, q_3	q_2, q_3
q_2, q_3	q_2, q_3	q_1, q_4	q_2, q_4, q_5	q_2, q_3	q_2, q_3
q_2, q_4	q_2, q_3	q_1, q_4	q_0, q_2, q_3	q_1, q_2, q_3, q_5	q_1, q_3, q_4, q_5
q_2, q_5	q_1	q_1	q_0, q_2, q_4	q_1, q_2, q_3, q_5	q_1, q_3, q_4, q_5
q_3, q_4	q_3	q_4	q_0, q_2, q_5	q_1, q_3, q_5	q_1, q_3, q_5
✓ q_3, q_5	q_3	q_4	q_0, q_3, q_4	q_1, q_3, q_5	q_1, q_3, q_5
q_4, q_5	q_3	q_4	q_0, q_3, q_5	q_1, q_3, q_5	q_1, q_3, q_5
q_0, q_1, q_2	q_1, q_2, q_5	q_1, q_3, q_5	q_0, q_4, q_5	q_1, q_3, q_5	q_1, q_3, q_5
q_0, q_1, q_3	q_1, q_3, q_5	q_3, q_4, q_5			
q_0, q_1, q_4	q_1, q_3, q_5	q_3, q_4, q_5			
q_0, q_1, q_5	q_1, q_5	q_3, q_5			

→ q_0 was initial state.

→ q_0, q_2, q_4, q_5 were acceptance states

Date: _____

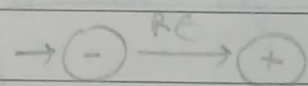
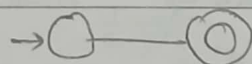


TG → Transition Graph.
GTG → Generalized TG
→ aa/ strings can go on transitions.
→ e. / can go on transitions.

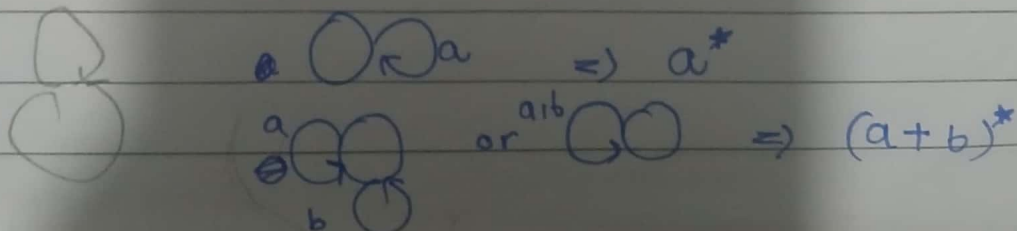
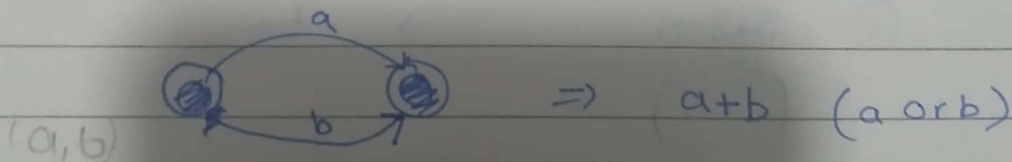
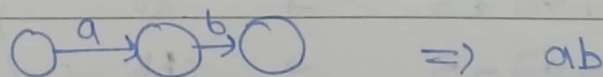
FA to Regular Expression.

GNFA → Generalized Nondeterministic Finite Automata.

- ① No outgoing from final/acceptance state. $\odot$ or $\oplus$
- ② No incoming to initial state $\ominus$ or $\ominus$



Some things that we know:



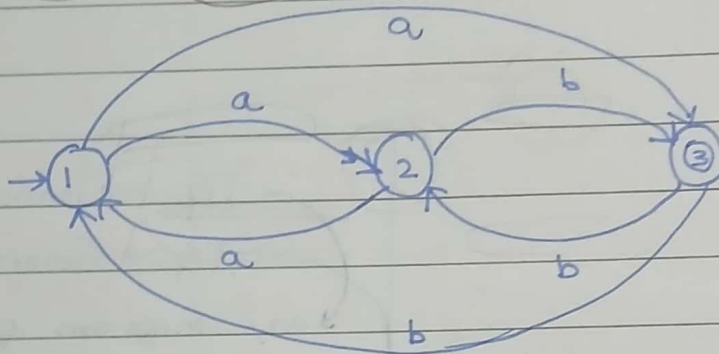
→ State Elimination

Date: _____

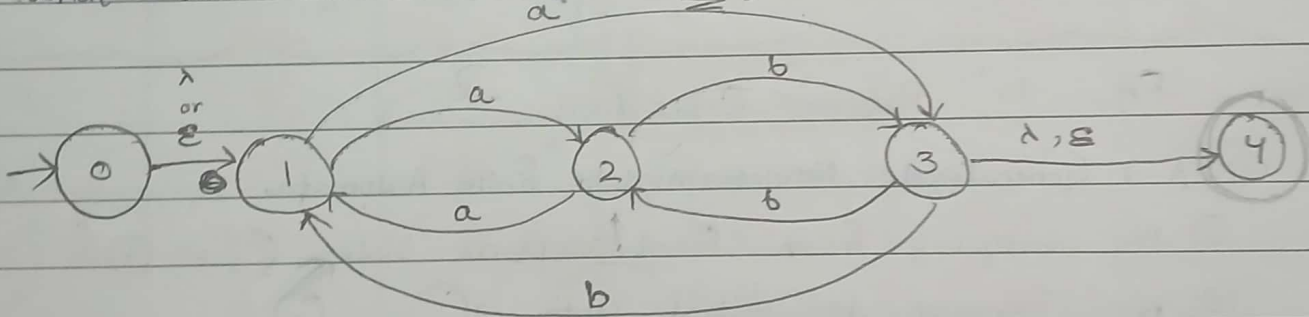
? → ? → ?

what is it taking & where is taking it, from where?

⇒ Example



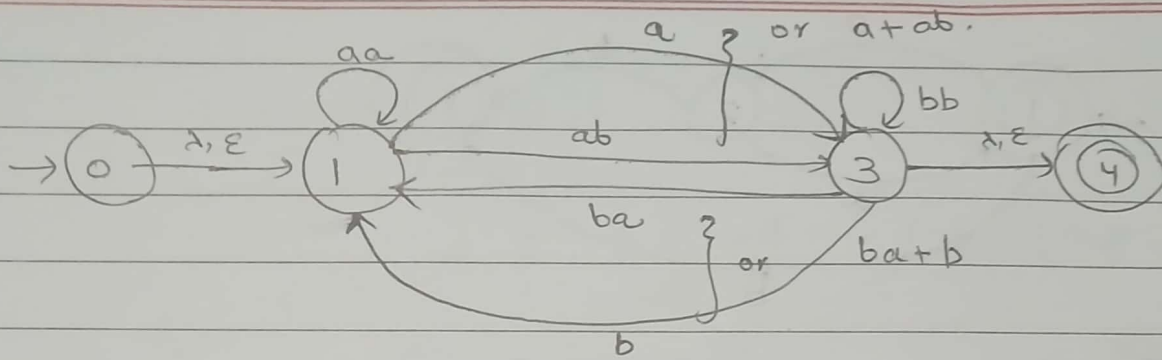
what will be the RE for this?



Before you eliminate a state → make a list:

Paths including 2 will result in strings:

- State:
- ① 1 → 2 → 1 → aa
 - ② 1 → 2 → 3 → ab
 - ③ 3 → 2 → 3 → bb
 - ④ 3 → 2 → 1 → ba



Paths including "3".

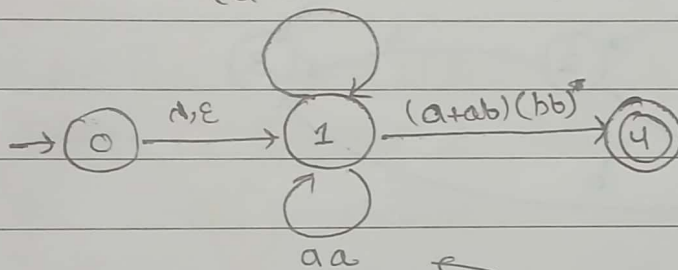
① $1 \rightarrow 3 \rightarrow 1$

$(a+ab)(bb)^*(b+ba)$

② $1 \rightarrow 3 \rightarrow 4$

$(a+ab)(bb)^*(\lambda \text{ or } \epsilon)$

$(a+ab)(bb)^*(b+ba)$



→ combine both loops

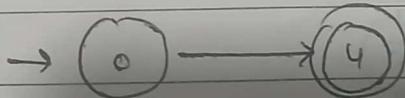
$aa + (a+ab)(bb)^*(b+ba)$

Now remove ①.

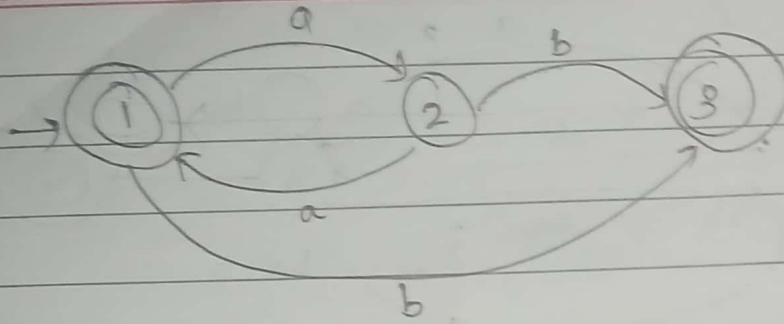
we get:

loop turns to $[\text{---}]^*$

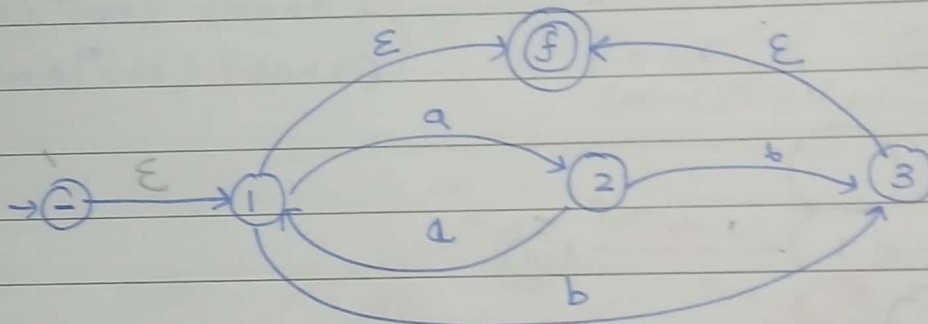
$[aa + (a+ab)(bb)^*(b+ba)]^* \cdot (a+ab)(bb)^*$ } regex



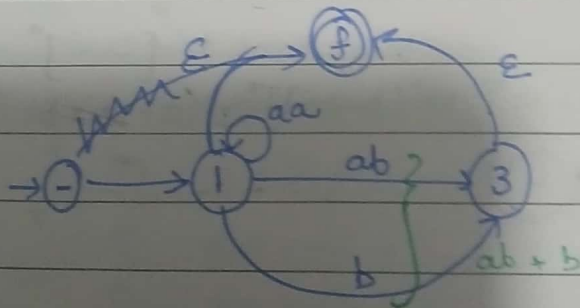
Date: _____



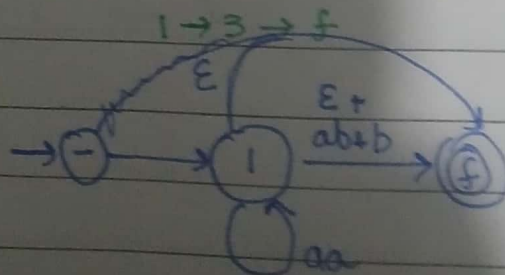
Two final states here, so first we make one final state.



Eliminate (2):
 $1 \rightarrow 2 \rightarrow 1 \rightsquigarrow aa$
 $1 \rightarrow 2 \rightarrow 3 \rightsquigarrow ab$



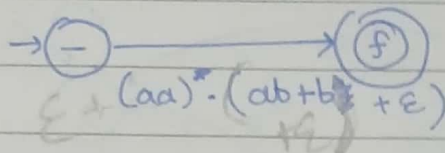
Eliminate (3):
 $1 \rightarrow 3 \rightarrow f \rightsquigarrow ab + b$



→ Eliminate ①:

- $\rightarrow 1 \rightarrow f$

$\leadsto (aa)^* \cdot (ab+ba)^* + \epsilon$



→ 12th Feb 2024

⇒ Properties of DFA

G Properties of Regular Languages.

→ Complement

→ union $\leadsto$ intersection

→ reverse

easy to show as
RE

→ concatenation → easy to show as
DFA / RE.

① Union & Intersection.

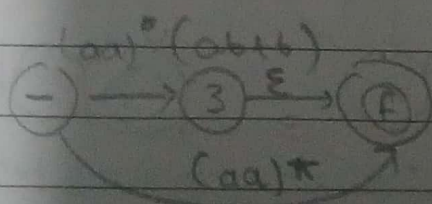
② Reverse & Complement

③ Concatenation & Kleene Star.

Eliminate ①

- $\rightarrow 1 \rightarrow 3 \Rightarrow (aa)^*(ab+ba)$

- $\rightarrow 1 \rightarrow f \Rightarrow (aa)^*$



Eliminate ③

- $\rightarrow 3 \rightarrow f$

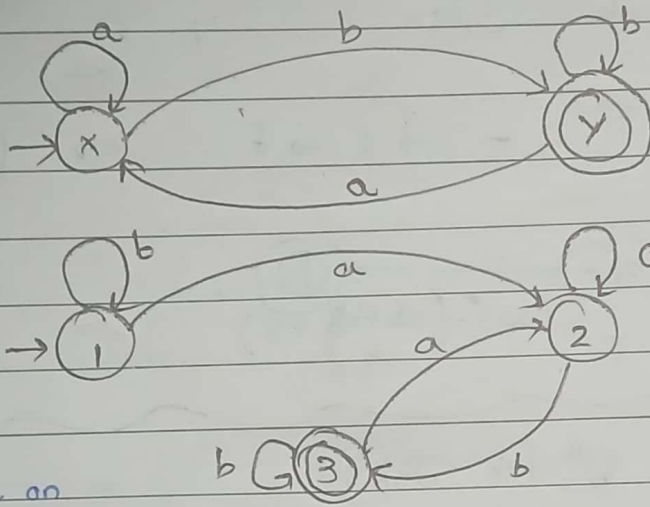
$(aa)^*(ab+ba) \cdot \epsilon$
+ $(aa)^*$

$(aa)^* [(ab+ba) + \epsilon]$

(i) UNION & INTERSECTION

 $\Sigma = \{a, b\}$
 $\&$
 $\Sigma = \{a, b\}$

need to be on
the same language set



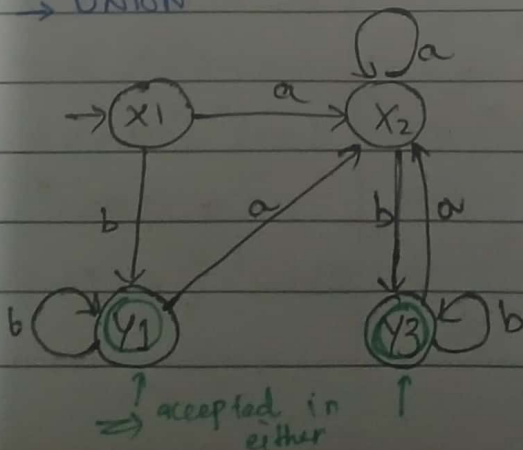
	a	b
combining $x \& y \rightarrow x1$ states	x2	y1
x2	x2	y3 y3
y1	x2	y1
y3	x2	y3

new states get added to the transition table.

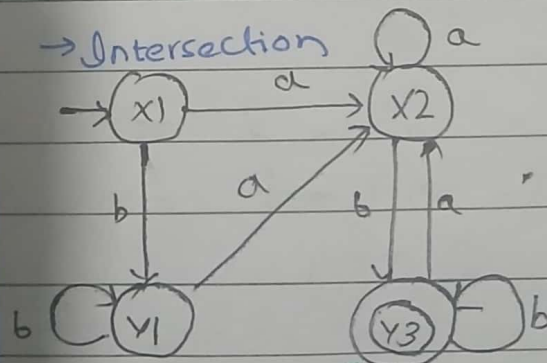
• for union $\rightarrow$ any state with y OR 3 will be accepting state.

• for intersection $\rightarrow$ we pick the state with y AND 3

$\rightarrow$ UNION



$\rightarrow$ Intersection

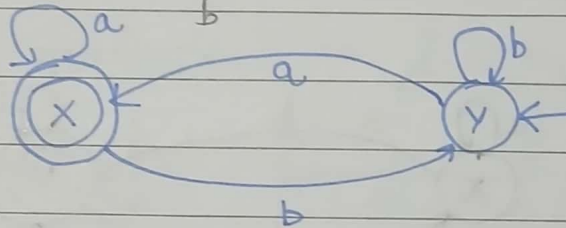
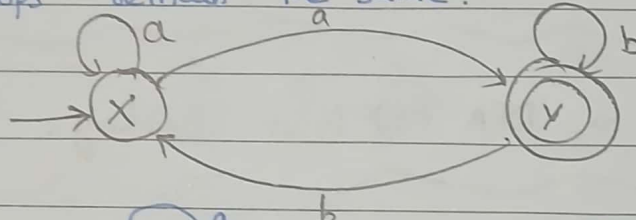


$\Rightarrow$ acceptance for intersection of 2 regular languages

② REVERSE & COMPLIMENT

Date: _____

- Initial becomes final. $\rightarrow \bigcirc \Rightarrow \bigcirc$
- Final becomes initial. $\bigcirc \Rightarrow \rightarrow \bigcirc$
- Transitions are reversed. $\rightarrow \bigcirc \rightarrow \bigcirc$
- Loops remain the same. $\bigcirc \leftarrow \bigcirc$

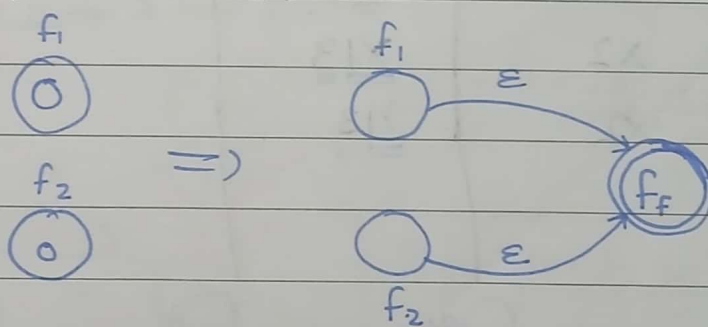


What do you do with GNFA?

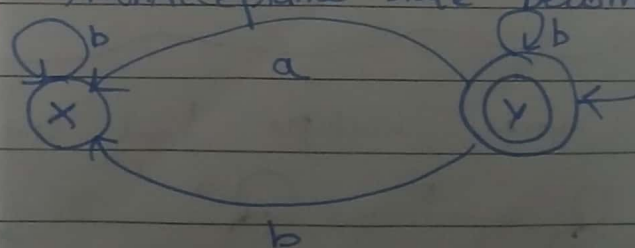
GTG?

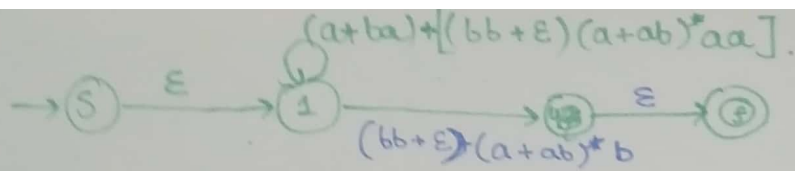
TGs $\Rightarrow$ Transition Graphs.

What if there are two final states? How do you convert them to the reverse.



For Compliment: $\rightarrow$ Acceptance state becomes non-acceptance
 $\rightarrow$ NonAcceptance state becomes Acceptance state.

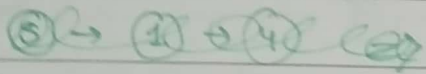




Date: _____

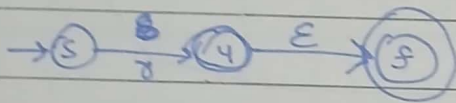
Eliminate (1) :

$$S \rightarrow 1 \rightarrow 4 \Rightarrow \epsilon [(a+ba)+[(bb+\epsilon)(a+ab)^*aa]]^* [(bb+\epsilon)(a+ab)^*b]$$



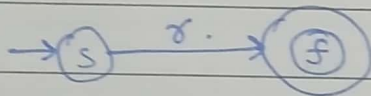
RE is:

$$r = [(a+ba)+[(bb+\epsilon)(a+ab)^*aa]]^* [(bb+\epsilon)(a+ab)^*b]$$



Eliminate (4) :

$$S \rightarrow 4 \rightarrow f. \Rightarrow r\epsilon \leftarrow \text{nothing new.}$$

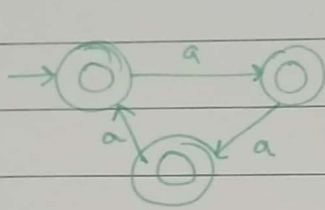


and r is :

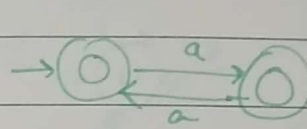
AFTER - MID TERM.

Date: 3rd March

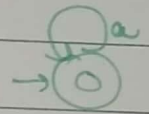
Minimization:



$$L_1 = \{a^*\}$$



$$L_2 = \{a^*\}$$



$$L_3 = \{a^*\}$$

Same language but different number of states.

How can you minimize number of states?

"Minimal" DFA : We get this after minimizing a DFA. It cannot be minimized further.

We have two types of states:



→ Accepting

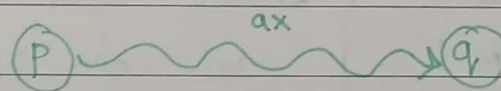
And any FSM cannot have any other state than these two



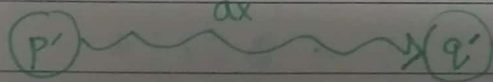
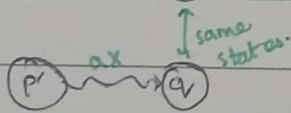
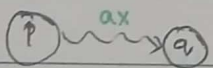
→ Non-Accepting

↪ verifies that a string is a part of a string

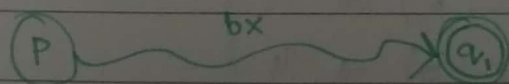
wavy line = multiple states in between.



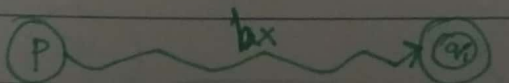
} both reach a rejection state on a string "ax"



then these are similar
can be minimized

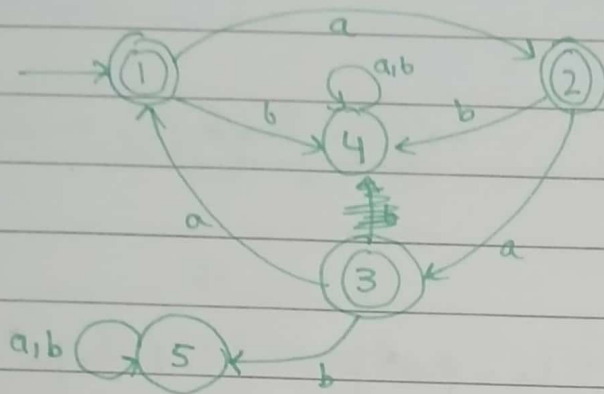


} both reach an acceptance state on a string "bx"



→ AUTOMATA MINIMIZATION EXAMPLE

Date: _____



→ first of all we can't remove state

compare states that are accepting.

→ Table Method: To reduce it with 1, 2, 3, 4 and 5 non-accepting.

Filling

no point in comparing with itself

no 1 here

	1	2	3	4
1		3, 1		
2				
3	2, 3		2, 1	
4				
5				3, 1 2, 3

no 5 here.

$1 \equiv 2 \equiv 3$
 $4 \equiv 5$

can compare 1 with 4 & 5 } because can't compare accepting
2 " 4 & 5 w/ non accepting
3 " 4 & 5 } 2 is dependent on 3 & 2

(Now, the transitions) $\Rightarrow (2, a) \rightarrow 3$ } this isn't similar yet
Comparing 1 & 2 $(1, a) \rightarrow 2$ }

with each other.

$(2, b) \rightarrow 4$
 $(1, b) \rightarrow 4$

these are similar

$(3, a) \rightarrow 1$

$(3, b) \rightarrow 5$

$(4, a) \rightarrow 4$
 $(5, a) \rightarrow 5$

$(4, b) \rightarrow 4$
 $(5, b) \rightarrow 5$
merging 4 & 5

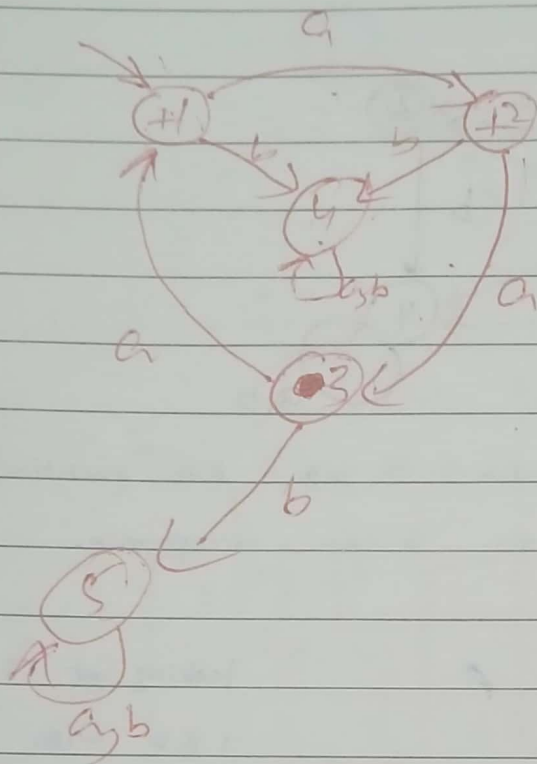
$(4, a) \rightarrow 4$
 $(4, b) \rightarrow 4$

$(4, a) \rightarrow 4$

once you reach self loop, you back track.

$\lambda, a, aaa, aaaa, aaaaaa$

Date: _____

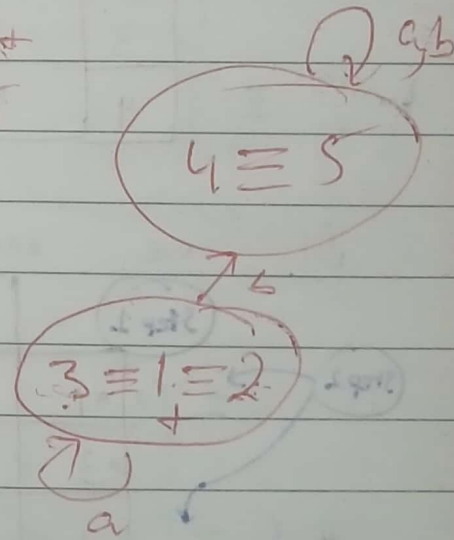
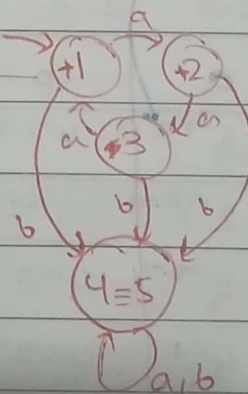


	1	2	3	4
1	α			
2	α	α		
3	α	α	α	α
4	α	α	α	α
5	α	α	α	α

$1, a \rightarrow 2+$
 $2, a \rightarrow 3+$
 $1, b \rightarrow 4$
 $2, b \rightarrow 4$
 $3, a \rightarrow 1+$
 $3, b \rightarrow 5$

$4, a \rightarrow 4-$
 $5, a \rightarrow 5$
 $4, b \rightarrow 4$
 $5, b \rightarrow 5$

	1	2	3	4
1	α			
2	α	α		
3	α	α	α	α
4	α	α	α	α
5	α	α	α	α



$1, a \rightarrow 2+$
 $1, b \rightarrow 4$

$2, a \rightarrow 3$
 $2, b \rightarrow 4$

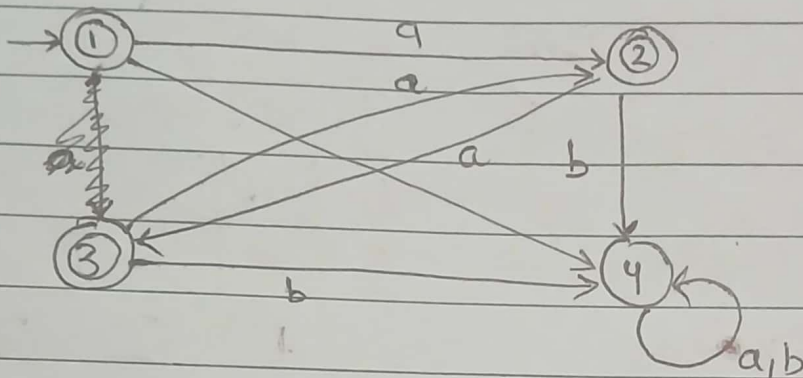
$3, a \rightarrow 1+$
 $3, b \rightarrow 5$

$4, a \rightarrow 4$
 $4, b \rightarrow 4$

$5, a \rightarrow 5$
 $5, b \rightarrow 5$

$4 \equiv 5$

Date: 5th March



Partitioning Method: We separate it into two partitions.
acceptance & non acceptance.

		a	b
Partition I {	1	2	4
	2	3	4
	3	2	4
Partition II {	4	4	4

looking at π for
1 2 3 with a
we have 2, 3, 2
reachable. which still
belongs to P-I

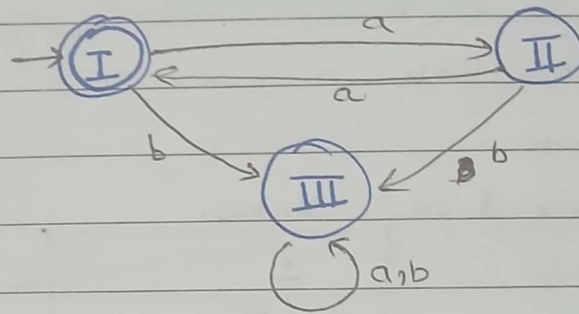
If we remove 2 as acceptance:

		a	b
I {	1	2 $\rightarrow$ I	4 $\rightarrow$ II
	3	2 $\rightarrow$ I	4 $\rightarrow$ I
II {	2	3 $\rightarrow$ I	4 $\rightarrow$ II
	4	4 $\rightarrow$ II	4 $\rightarrow$ II

update this based on updated partitions
since 3 & 4 are different states, we need to split the group. (If we don't split it, it'll become non-deterministic)

Step 6 $\rightarrow$ draw automata:

Date: 5th April



Ex:

From Activity:

①

Table filling method:

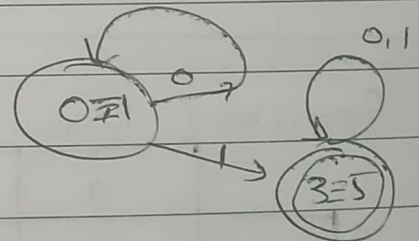
	0	1	2	3	4
1	0,1				
2	X	X			
3	X	X	X		
4	X	X	X	X	
5	X	X	X	✓	X

acceptance states are:

q_3, q_5

① Separate/cross out stuff you can't compare/check for (accepting w/ unaccepting) Done in green.

② Then you make the transitions:



$1,0 \rightarrow 1$ $1,1 \rightarrow 3$ $2,0 \rightarrow 1$

$2,0 \rightarrow 1$ $2,1 \rightarrow 4$

$0,0 \rightarrow 1$ $0,1 \rightarrow 3$ $1,0 \rightarrow 0$

$0,1 \rightarrow 3$

$1,1 \rightarrow 3$

$2,0 \rightarrow 1$

$2,1 \rightarrow 4$

$3,0 \rightarrow 5$

$3,1 \rightarrow 5$

accepting w/ nonaccepting $\Rightarrow X$

$4,0 \rightarrow 3$

$4,1 \rightarrow 3$

$5,0 \rightarrow 5$

$5,1 \rightarrow 5$

$3 \equiv 5$

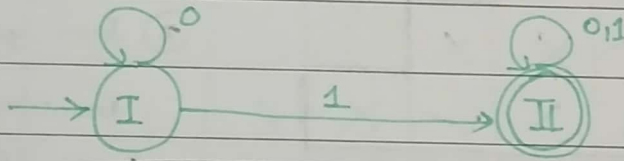
Ex: ① Partition Table:

Step ① Make Transition table

Step ② Make Partitions.

Step ③ Remove Unreachable states.

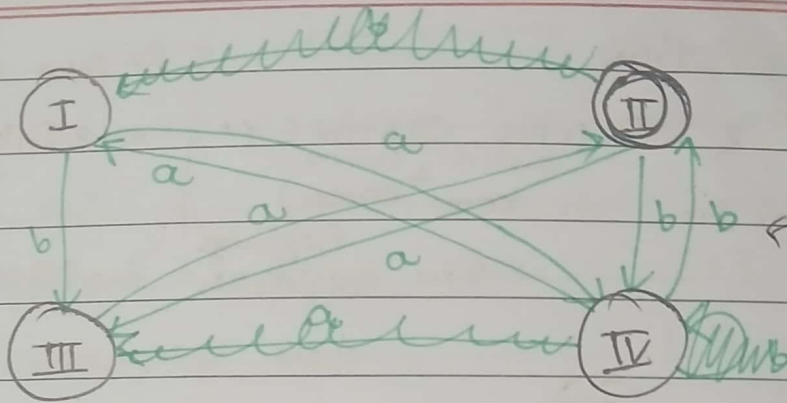
		0	1	
I {	q_0	$q_1 \rightarrow I$	$q_3 \rightarrow II$	These must be → same vertically, otherwise you need to split it.
	q_1	$q_0 \rightarrow I$	$q_3 \rightarrow II$	
	q_2	q_1	q_4	un reachable.
	q_4	q_3	q_3	
II {	$*q_3$	$q_5 \rightarrow I$	$q_5 \rightarrow I$	q_1 & q_3 still remain reachable even after removing 2 & 4
	$*q_5$	$q_5 \rightarrow I$	$q_5 \rightarrow I$	



Example #2: Partition Table:

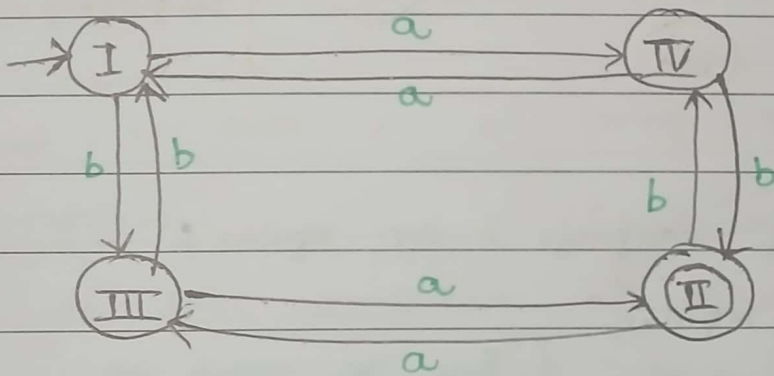
		a	b			a	b
I {	1	2 ^I	3 ^I	II {	*4	6 ^{III}	8 ^{IV}
	2	5 ^I	4 ^{II}		1	2 ^{IV}	3 ^{III}
	3	4 ^{II}	9 ^I		5	2 ^{IV}	6 ^{III}
	5	2 ^I	6 ^I		7	8 ^{IV}	6 ^{III}
II {	6	4 ^{II}	7 ^I	III {	9	8 ^{IV}	3 ^{III}
	7	8 ^I	6 ^I		3	4 ^{II}	9 ^I
	8	7 ^I	4 ^{II}		6	4 ^{II}	7 ^I
	9	8 ^I	3		2	5 ^I	4 ^{II}
III {	4	6 ^I	8 ^I	IV {	8	7 ^I	4 ^{II}

Date: _____



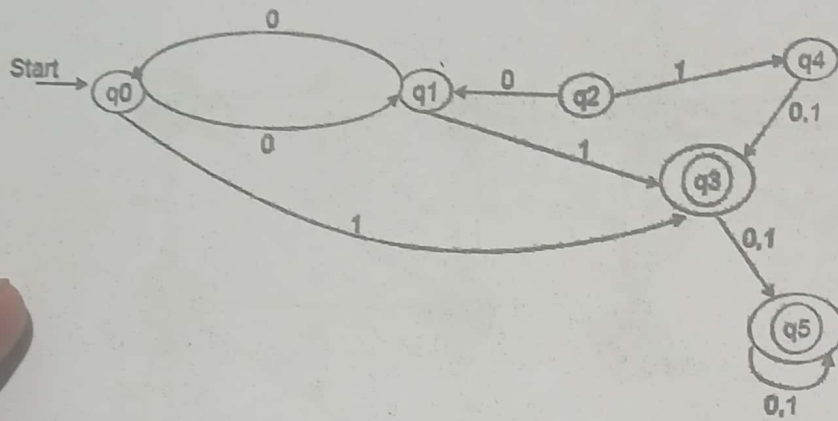
Looks terrible.

So here's a cleaner version.

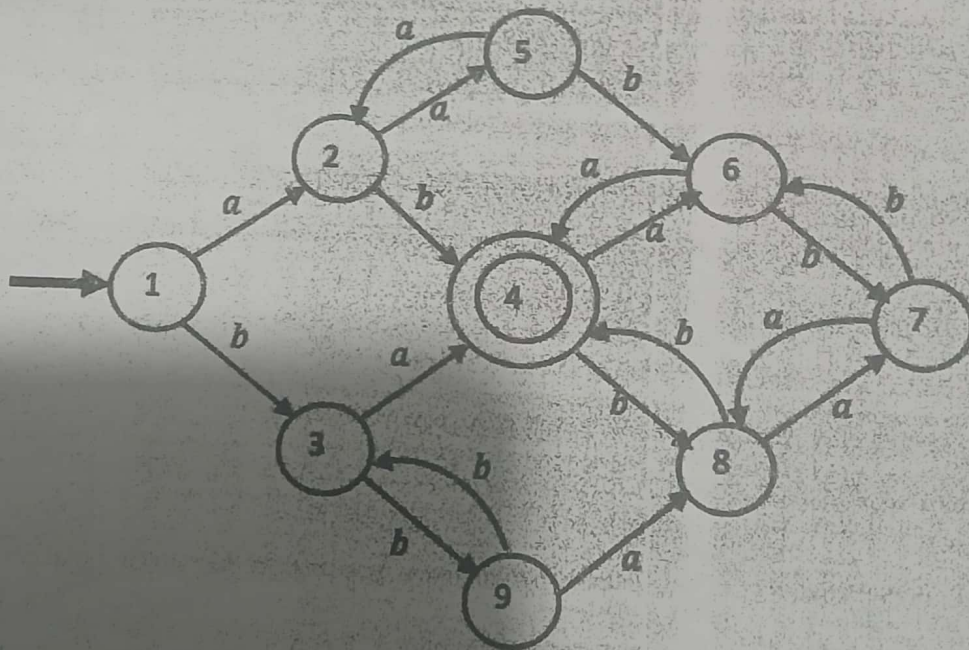


For the following DFA's find the minimal DFA which must accept the same language as the original DFA.

1.

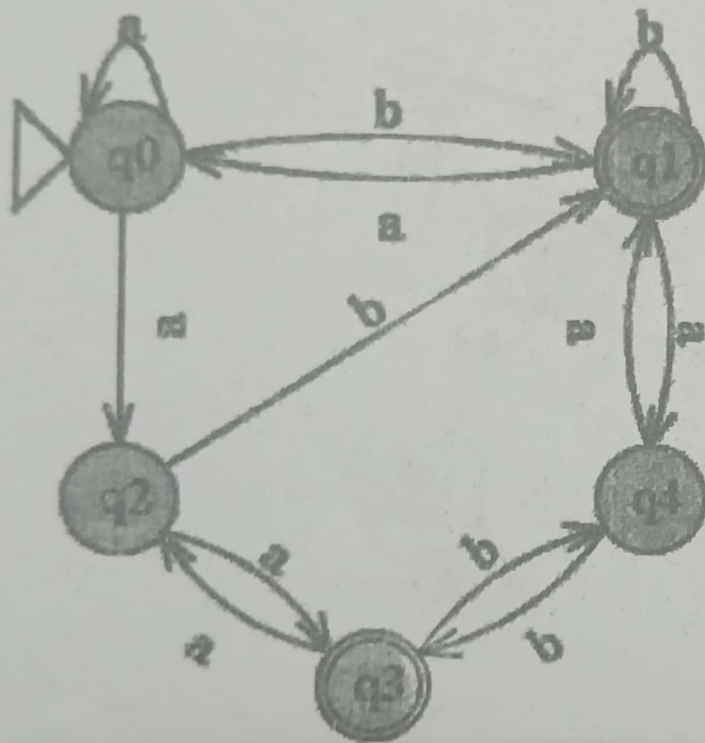


2.

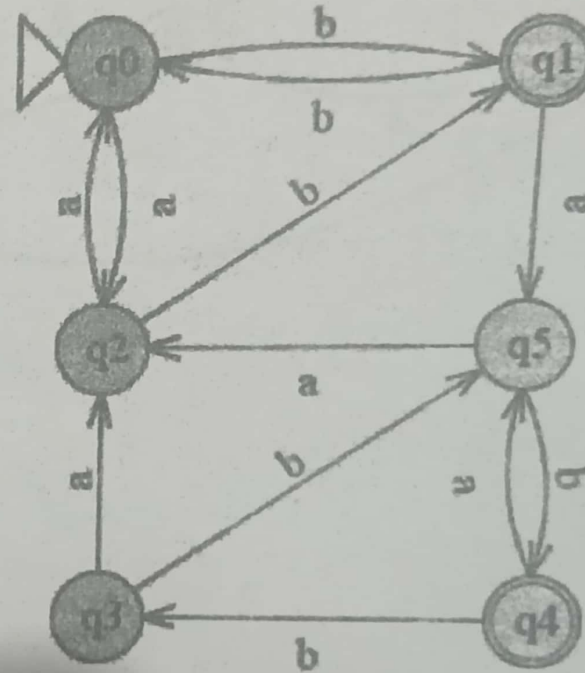


Question 3: Perform DFA Minimization.

FA 1



FA2



PROPERTIES OF REGULAR LANGUAGES

Date: 10th March

① UNION & INTERSECTION

② CONCATENATION & KLEENE CLOSURE (*)

③ COMPLIMENT (Kleene Star v/s Kleene Closure)

} All these properties result in a REGULAR LANGUAGE.

These properties are "closed" under these operations.

FA

NFA

DFA

RE

} all result in a regular language / Identify a RL.

When:

Language 1 & Language 2 are regular:

① Union would be regular.

② Intersection " " " "

③ Concatenation " " " "

④ Kleene Closure " " " "

⑤ Compliment " " " "

} They can reach an acceptance state.

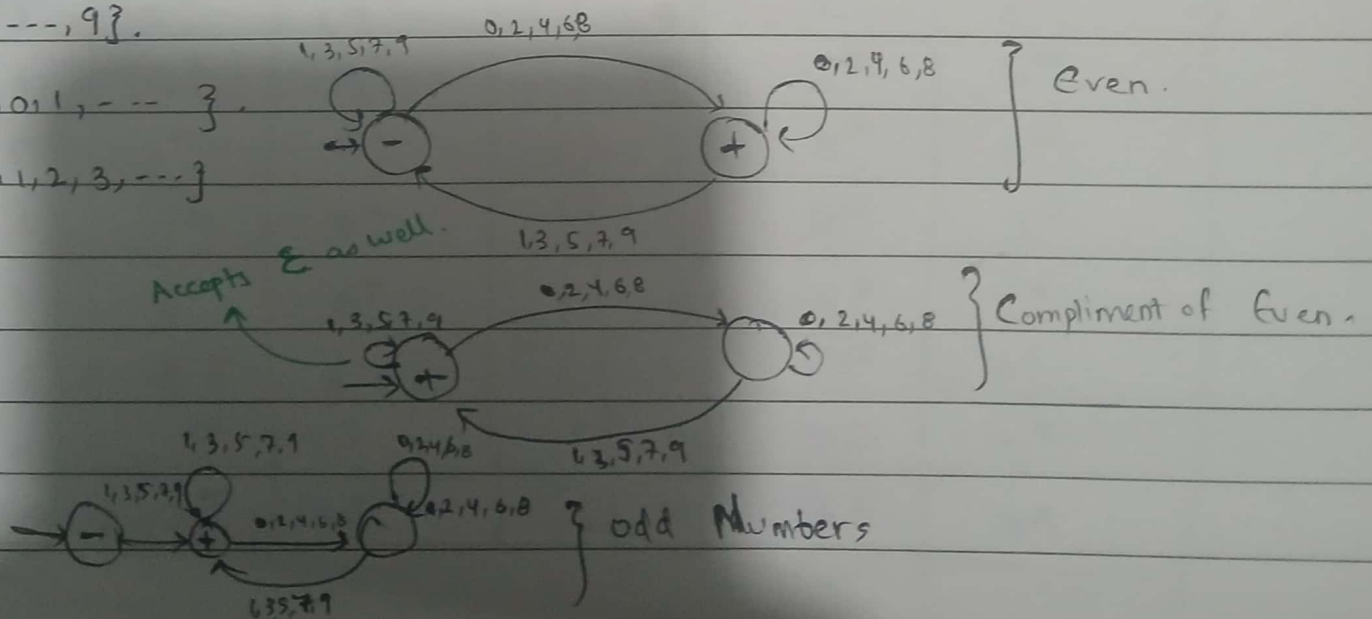
Intersection of Even Set / Odd Set $\Rightarrow$ Empty Set or ϕ

Compliment of Even Set = Odd Set (+ ϵ) || They're not compliments of each other.

$\Sigma = \{0, \dots, 9\}$.

$\Sigma^* = \{1, 0, 1, \dots\}$.

$\Sigma^+ = \{0, 1, 2, 3, \dots\}$



Date: 10th March

This is relevant when working with electronics.

→ If there is a delay / no signal $\Rightarrow$ what should a machine do?

$\Rightarrow$ In terms of a digital clock. 1:3 will be shown as 01:03

1:13 $\Rightarrow$ 01:13 { 00:01

$\Rightarrow$ Whereas if I had to write 0, I would never be writing 0000 etc in real life

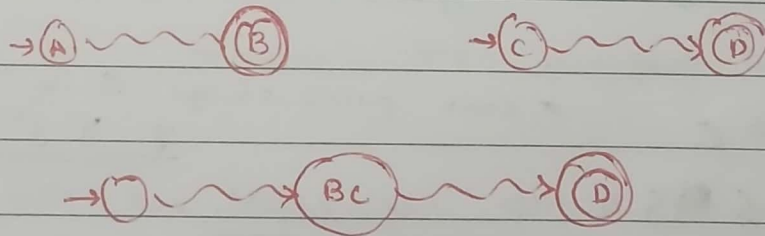
Union: Even $\cup$ odd $\Rightarrow$ all strings. // both states will become accepting ???

Intersection:

(No) cuz can't accept epsilon.

When we take intersection of Even & odd, there is nothing in common. i.e. no accepting state.

CONCATENATION:



→ Pumping Lemma

Date: 12th March

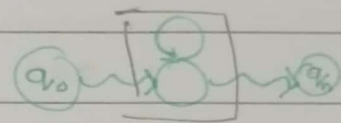


Number of States $N =$	2	3	4
Length of string $ S =$	1	2	3

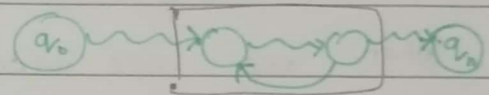
$$\left. \begin{array}{l} |S| = N-1 \\ \text{or } |S| < N \end{array} \right\} \begin{array}{l} \nearrow \\ \text{assuming no loops.} \end{array}$$

If there are any loops:

$$|S| \geq N$$



OR:



Remember Pigeonhole Principle

Now, moving onto the lemma:

$$\text{If: } |S| \geq N,$$

Then: There exists some $xyz = S$ (you can split S into xyz)

Such That:

$$\textcircled{1} \quad y \neq \epsilon \quad \text{can't pump strings if it's } \epsilon$$

$$\textcircled{2} \quad |xy| \leq N$$

$$\textcircled{3} \quad \text{For all } n \geq 0, \text{ if } xy^n z \in L$$

then L is regular

Example:

$$L = \{ ab(ab)^n \mid n \geq 0 \}$$

$$L = \{ ab, \underline{abab}, ababab, \dots \}$$

Pick a string ≥ 2 in length, so we can split into xyz

since

$$xy^0z = \epsilon(ab)^0\epsilon = \epsilon\epsilon\epsilon$$

not a part of language.

$$(ab)^*ab = a(ba)^*b = ab(ab)^*$$

Date: 12th March

$$abab \rightsquigarrow xyz$$

$$x = a$$

$$y = ba$$

$$z = b$$

// cannot be ϵ

$$ab$$

$$ab$$

$$\epsilon$$

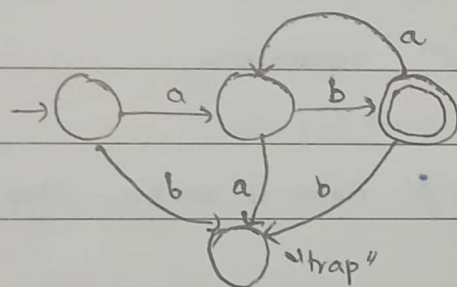
$$xy^0z = ab, xy^1z = abab, xy^2z = ababab$$

$$\text{for } (ab)^* \rightarrow x = \epsilon, y = ab, z = \epsilon$$

y is the thing that is pumping out strings.

$$y^0 = \epsilon, y^1 = ab, y^2 = abab$$

DFA for $ab(ab)^*$:



If a language is regular, then pumping lemma can prove it.

For something with two loops, you split it into two, then apply pumping lemma for each part separately.

For something like: $(ab)^*(ba)^*$

$$L = \{ (ab)^n(ba)^m \mid n, m \geq 0 \}$$

$\rightarrow ab$ & ba are independent, for n & m

Regular Grammars

Date: 19th March 2025

"Grammars" → Rules for a language.

Pumping lemma helped us prove that a language is regular.

$$\{ a^n b^n \mid n \geq 0 \} \quad L = \{ \lambda, ab, aabb, aaabbb, \dots \}$$

• number of a's & b's are reliant on each other.

• in pumping lemma, we could split the R.E. into chunks & apply the PL to it. But in case of $a^n b^n$ — n is shared by both, so we can't split it. If it was $a^n b^m$, we could split to a^n & b^m & then apply PL to a^n & b^m separately.

Another one:

$$a^n b^m \mid n \geq m \quad // \quad n \geq m \text{ is another dependency.}$$

↳ not regular

$$L = \{ a, b, aab, aa, aaaaabb, \dots \}$$

Let's say we pick $aabb$: & $x=a$, $y=ab$, $z=b$

$$\begin{array}{lcl} xy^n z & n=0 & \rightarrow x y^0 z = a(ab)^0 b = ab \in L \quad \checkmark \\ & n=1 & \rightarrow x y^1 z = a(ab)^1 b = aabb \in L \quad \checkmark \\ & n=2 & \rightarrow x y^2 z = a(ab)^2 b = aabab b \notin L \quad \times \end{array}$$

what if we said: $x=a$, $y=a$, $z=b$

$$\begin{array}{lcl} n=0 & \rightarrow & xy^0 z = a(a)^0 b = ab \in L \quad \checkmark \\ n=1 & \rightarrow & xy^1 z = a(a)^1 b = aab \in L \quad \checkmark \\ & & xy^2 z = a(a)^2 b = aaab \in L \quad \checkmark \end{array}$$

PUMPING LENGTH:

→ Pumping Lemma can't pump non-regular languages, but it cannot identify them!

Let's consider palindromes.

$$\begin{array}{ccc} a & b & c \\ \hline W \end{array} \quad \begin{array}{ccc} c & b & a \\ \hline WR \end{array}$$

→ ALL palindromes upon $\{0,1\}$

$L = \{ \epsilon, 0, 1, 00, 11, 101, 010, 111, 000, 1111, 0000, 1001, 0110, 10001, 10101, 01110, 01010, \dots \}$

let's pick this:

$x = 1, y = 010, z = 1$ / can pump 010 to make many palindromes, but not all!!

→ While it's suggesting it might be regular, it isn't!!

→ Can't write a R.E for palindromes. | or make a DFA for it.

→ PUMPING LEMMA FAILS HERE

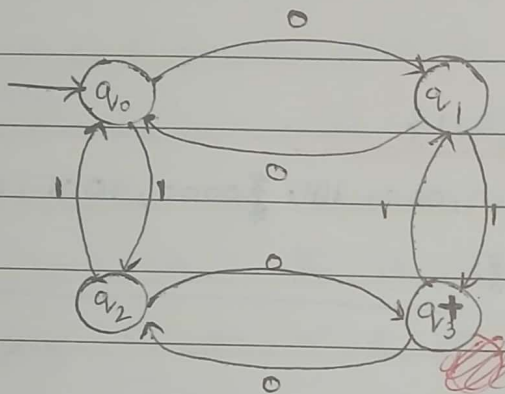
Regular Grammar :

 $V \rightarrow$ variables $A, B, A, B, \dots$ $T \rightarrow$ terminals. $\leq U \leq$

Start Variable

 $\in V$

Productions / Rules



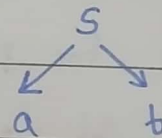
✓ Even 1s & 0s.

language: $L = \{a\}$.mar: $G = S \rightarrow a$

f%

$$\begin{array}{c} S \\ \swarrow \\ a \end{array} \} \text{tree.}$$
 $L = a+b$ $G \Rightarrow S \rightarrow a | b // a \text{ or } b$

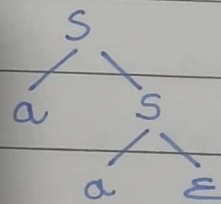
Tree:



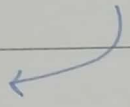
oops): If

 $L = a^*$ $G \Rightarrow S \rightarrow aS | \epsilon \text{ or } \lambda$

Tree

 $\Rightarrow aa\epsilon$

Now for the DFA



$$q_0 \rightarrow 1 q_2 \mid 0 q_1$$

$$q_1 \rightarrow 1 q_3 \mid 0 q_0$$

$$q_2 \rightarrow 1 q_0 \mid 0 q_3$$

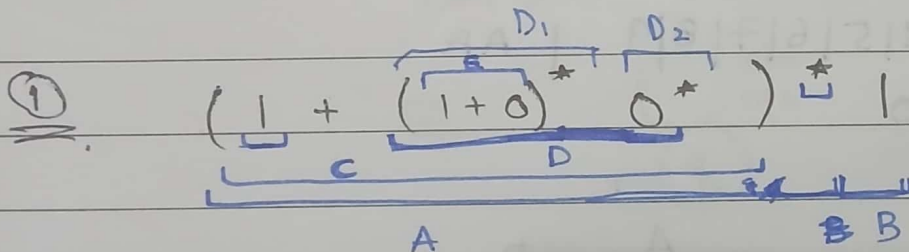
$$q_3 \rightarrow 1 q_1 \mid 0 q_2 \mid \underline{\underline{\epsilon}}$$

on the final you add
 ϵ

R.E.

$$\textcircled{1} (1 + (1+0)^* 0^*)^* 1$$

$$\textcircled{2} (a+b)(ab)(a+b)^*(ba)$$



start var = S.

$$S \rightarrow AB$$

$$B \rightarrow 1 \quad // \text{terminal.}$$

$$A \rightarrow CA \mid \epsilon$$

$$C \rightarrow B \mid D$$

// B since $1 + D$ & 1 is B.

$$D \rightarrow D_1 D_2$$

// since concatenation.

$$D_1 \rightarrow \epsilon D_1 \mid \epsilon$$

$$D_2 \rightarrow 0 D_2 \mid \epsilon$$

// terminal D_2 or ϵ

$$\epsilon \rightarrow 0 \mid 1$$

// 0 or 1

$$(2) \quad \underbrace{(a+b)}_A \underbrace{(ab)}_B \underbrace{(a+b)^*}_C \underbrace{(ba)}_D$$

Start var $\Rightarrow S$.

$$S \rightarrow ABCD$$

$$A \rightarrow a | b$$

$$B \rightarrow ab$$

$$C \rightarrow AC | \epsilon$$

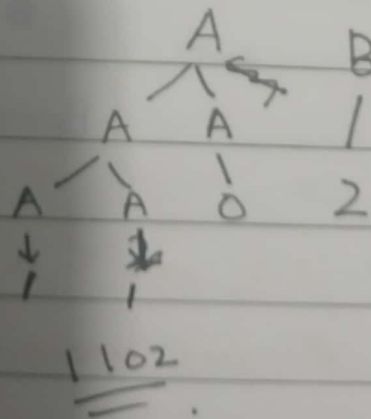
$$D \rightarrow ba$$

• Events

$$S \rightarrow AB$$

$$A \rightarrow 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 \quad | \quad AA$$

$$B \rightarrow 2 | 4 | 6 | 8 | 0$$



MEALY & MOORE MACHINES

Date: 24/3/25

→ IRREPRODUCIBLES

→ automata with input & outputs.

→ Machines with outputs:

DFA/NFA are acceptors.

Traverses an input, but also gives some output.

Suppose:

Complement of a binary string / binary language.

complement (0) → 1

"" (1) → 0

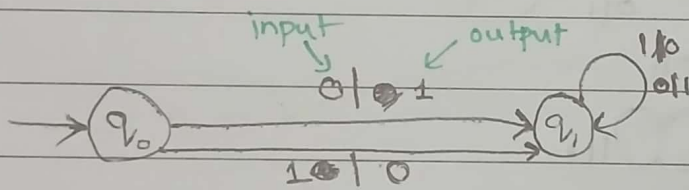
OR:

Capitalizing a string of small alphabet.

MEALY

Two Ways To GENERATE OUTPUTS.

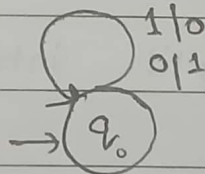
Mealy



⇒ One input will only have one output

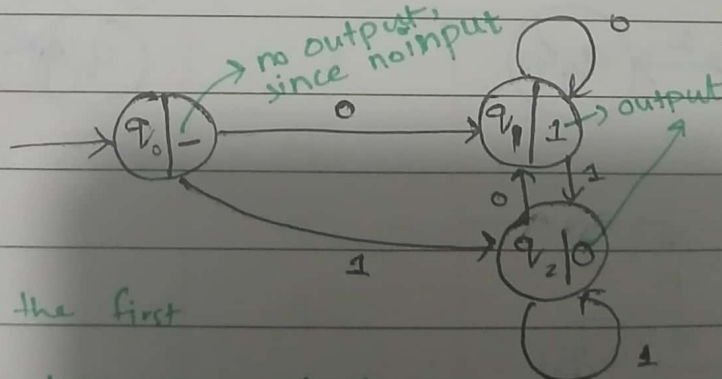
final states are not essential here.

OR



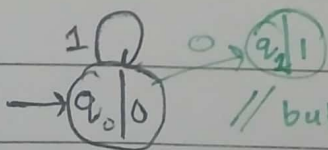
MOORE

outputs rest on a state



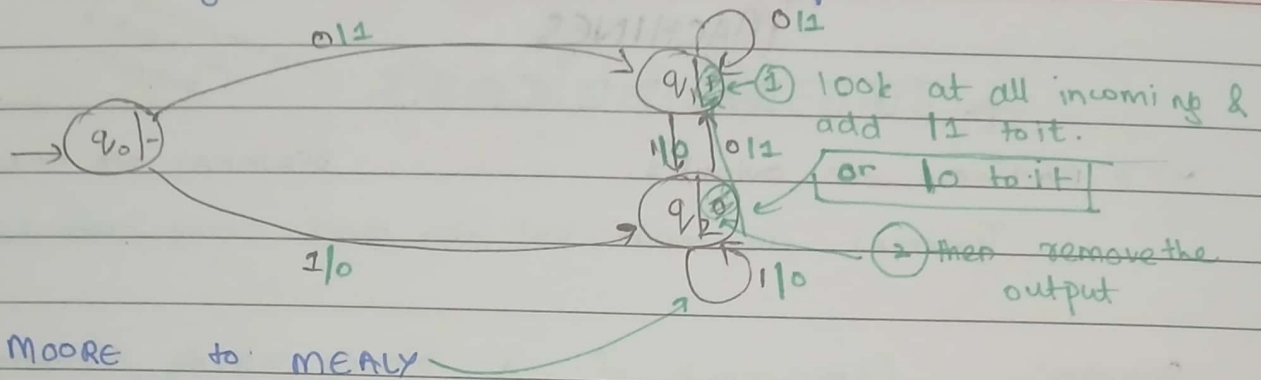
If there's a loop on the first

state, then it can have an output

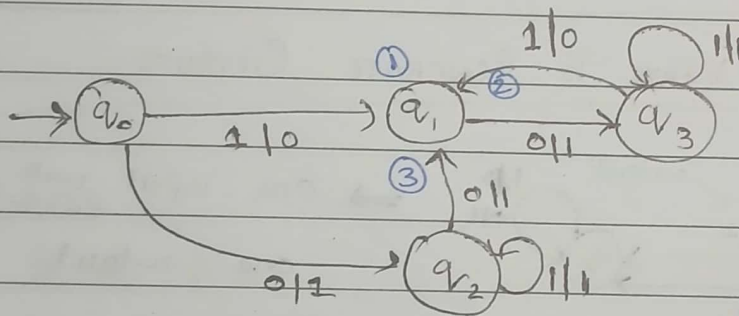


// but this causes ambiguity cause what if I have 0 as input. It'll still give output as "0", when it shouldn't.

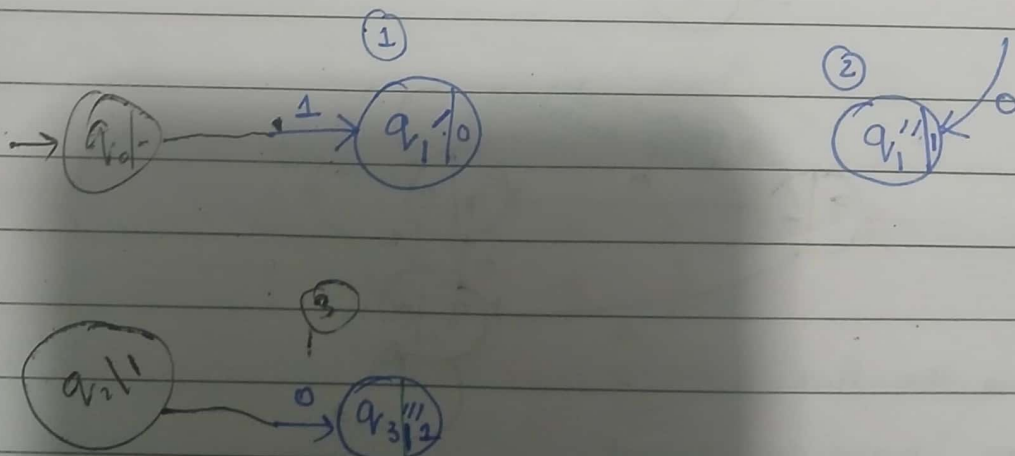
Both Mealy & Moore Machines are Equivalent. Date: _____

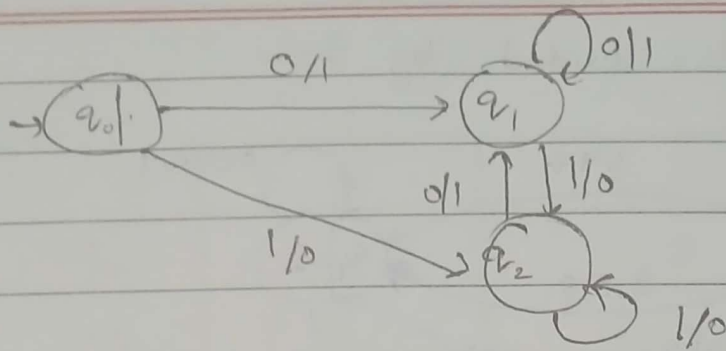


MEALY To MOORE.



for q_1 : split into states for each incoming:



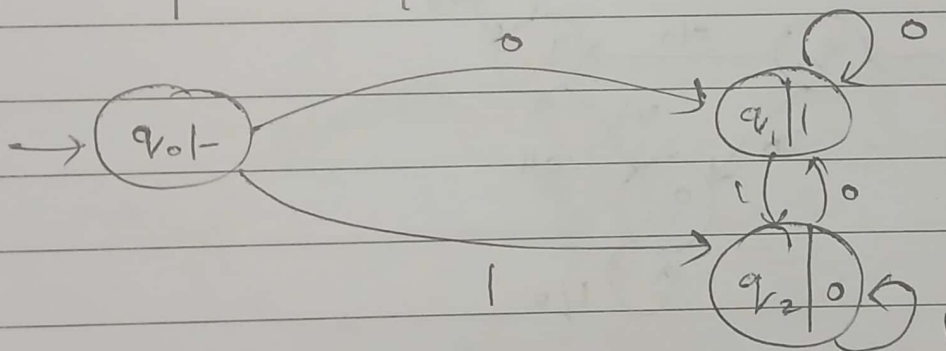


⇒ Transition Table: (MEALY)

	0	1
q ₀	q ₁ 1	q ₂ 0
q ₁	q ₁ 1	q ₂ 0
q ₂	q ₁ 1	q ₂ 0

⇒ to moore

	0	1	O/P
q ₁	q ₁ 1	q ₂ 0	1
q ₂	q ₁ 1	q ₂ 0	0
q ₀	q ₁ 1	q ₂ 0	0



⇒ Transition Table: (MOORE)

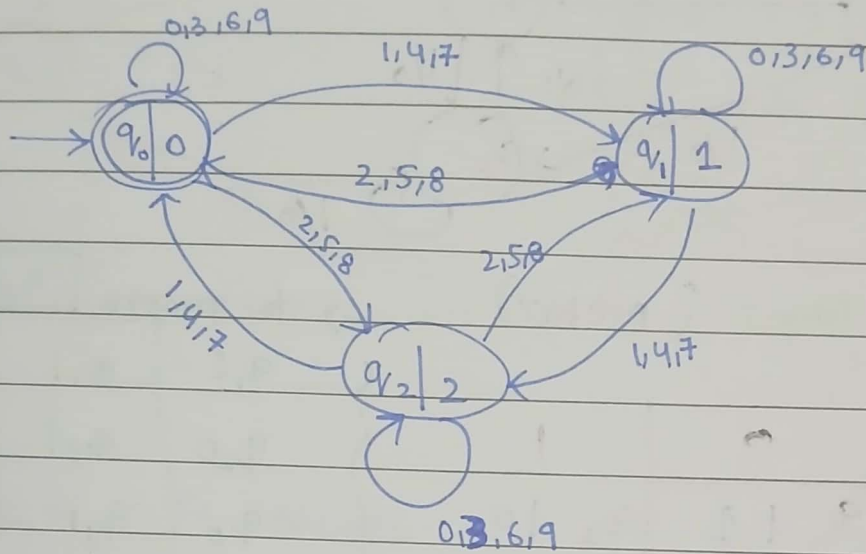
	0	1	O/P
q ₀	q ₁	q ₂	-
q ₁	q ₁	q ₂	1
q ₂	q ₁	q ₂	0

when converting to mealy:
 1 → all transitions to q₁ will generate 1
 0 → all transitions to q₂ will generate 0

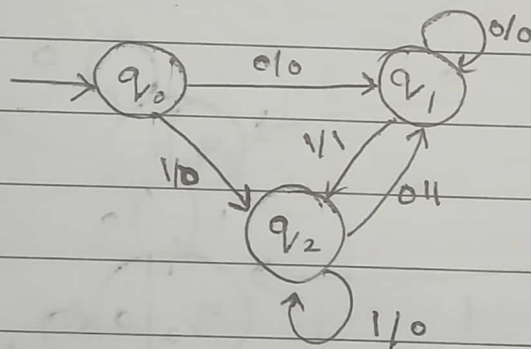
MEALY TO MOORE

Date: _____

Remainder of # divided by 3.



→ Mealy to Moore Conversion



⇒ Transition Table: (Mealy)

	0	1
q ₀	q ₁ / 0	q ₂ / 0
q ₁	q ₂ / 0	q ₂ / 1
q ₂	q ₁ / 1	q ₂ / 0

If q₀/q₁/q₂ appear with more than one output we split them for each output.

q₁0 and q₁1 } q₂0 & q₂1

or 3200M
YJAJM

Date: _____

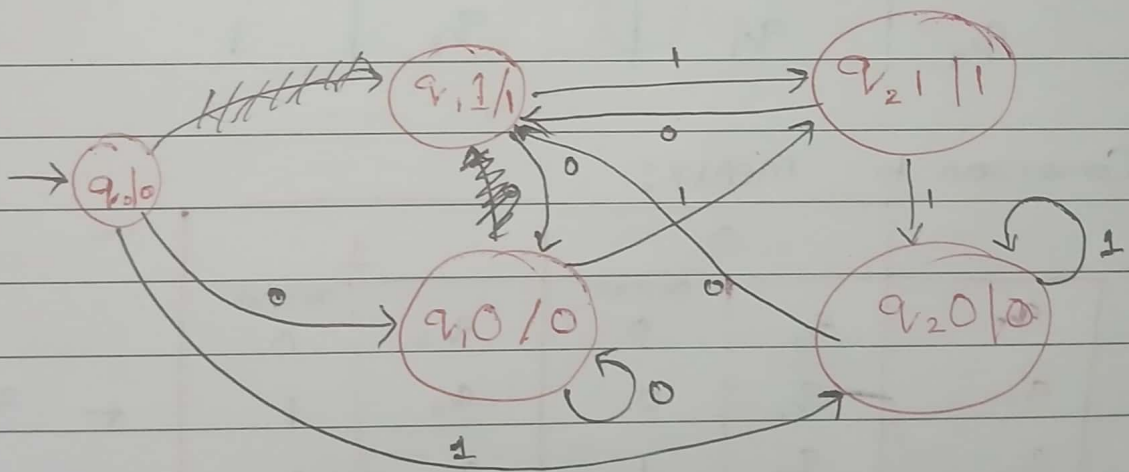
Make a TT with all new states + states that you didn't split.

	0	1	output
q_0	$q_{1,0}$	$q_{2,0}$	0
$q_{1,1}$	$q_{1,0}$	$q_{2,1}$	1
$q_{1,0}$	$q_{1,0}$	$q_{2,1}$	0
$q_{2,1}$	$q_{1,1}$	$q_{2,0}$	1
$q_{2,0}$	$q_{1,1}$	$q_{2,0}$	0

- if you don't want it to be 0 or 1

output is always the last digit of state.

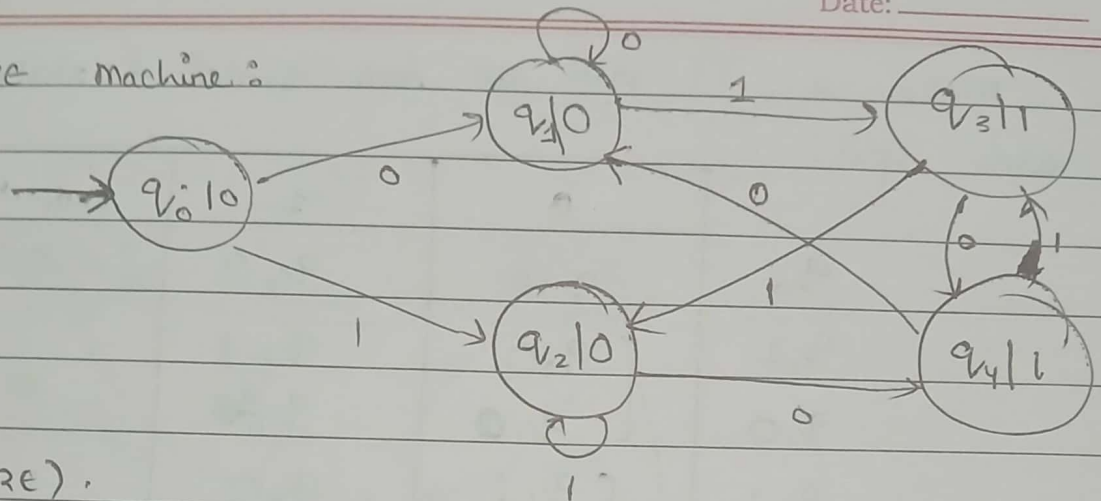
Copy the exact thing to these.



MOORE TO MEALY

Date: _____

Same MOORE machine :



T.T (MOORE).

	0	1	Output
q_0	q_1	q_2	0
q_1	q_1	q_3	0
q_2	q_4	q_2	0
q_3	q_4	q_2	1
q_4	q_1	q_3	1

Conversion to MEALY:

	0	OUTPUT	1	OUTPUT
q_0	q_1	0	q_2	0
q_1	q_1	0	q_3	1
q_2	q_4	1	q_2	0
q_3	q_4	1	q_2	0
q_4	q_1	0	q_3	1

$q_1 \xrightarrow{0} q_1(0)$
 $q_1 \xrightarrow{1} q_3(1)$
 $q_4 \xrightarrow{0} q_1(0)$
 $q_4 \xrightarrow{1} q_3(1)$

Can merge q_1 & $q_4 \Rightarrow q_{14} \xrightarrow{0} q_{14}(0)$

$\xrightarrow{1} q_{23}(1)$

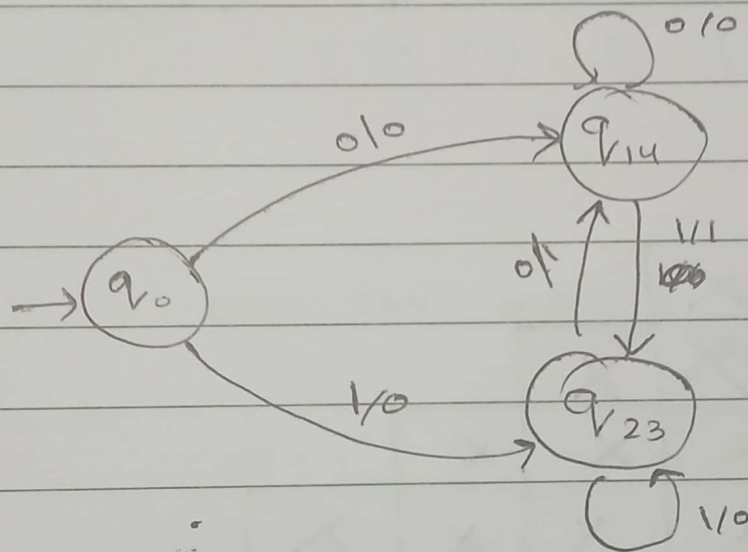
Can also merge q_2 & $q_3 \Rightarrow q_{23}$

$\xrightarrow{0} q_{14}(1)$

$\xrightarrow{1} q_{23}(0)$

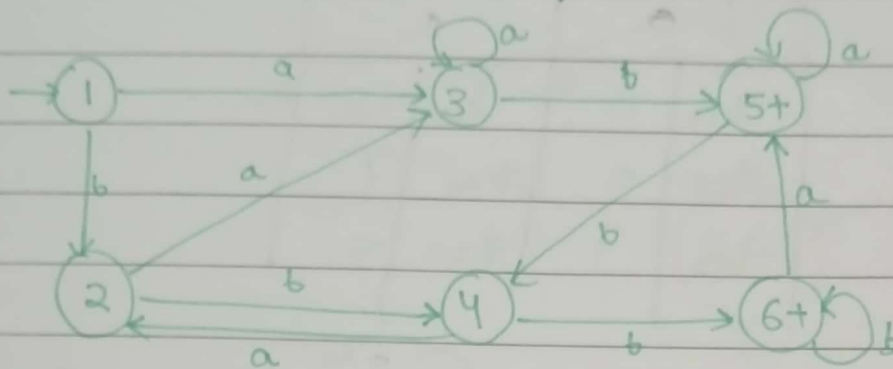
Updated Table:

	0	1
q_0	q_{14} 0	q_{23} 0
q_{14}	q_{14} 0	q_{23} 1
q_{23}	q_{14} 1	q_{23} 0



Another Minimization Example

Date: _____



2	1,3				
3	X	X			
4	X	1,2	X		
5	X	X	X	X	
6	X	X	X	X	X
	1	2	3	4	5

Compare(1,2) → $\delta(1,a) \rightarrow 3$ $\delta(2,a) \rightarrow 3$ (3,3)
 $\delta(1,b) \rightarrow 2$ $\delta(2,b) \rightarrow 4$ (2,4)

Compare(1,3) $\delta(3,a) \rightarrow 3$ $\delta(1,a) \rightarrow 3$ (3,3)
 $\delta(3,b) \rightarrow 5+$ $\delta(1,b) \rightarrow 2$ (5+,2) // can't merge.

Compare(2,3) $\overset{a}{(3,3)}$ & $\overset{b}{(4,5+)} \rightarrow$ // can't merge.

compare(1,4) $\delta(1,a) \rightarrow 3$ $\delta(4,a) \rightarrow 2$ (3,3)
 $\delta(1,b) \rightarrow 2$ $\delta(4,b) \rightarrow 6+$ (2,6+) X.

compare(2,4) (3,2) (4,6+) ← X.

compare $(4, 3) : (3, 2) \quad (5+, 6+)$.

$\hookrightarrow 3, 2$ is X

Compare $(5+, 6+) :$

$\delta(5, a) \rightarrow 5+$	}	$\delta(5, a) \rightarrow 5+$
$\delta(5, b) \rightarrow 4$		$\delta(6, b) \rightarrow 6+$

$(5+, 5+) \quad \& \quad (4, 6+) \rightarrow X.$

OOPS, CAN'T MINIMIZE :(

CFG for $a^n b^m$ $n \geq m$. m at min = 0

can only cater for $n+1$
 n at $\min = 1$.

$$S \rightarrow (A \mid aSB$$
$$S \rightarrow aA \mid a$$
$$\left\{ \begin{array}{l} aaaaab \\ aaaaabb \end{array} \right\} \quad \left\{ \begin{array}{l} aab \\ aaabb \end{array} \right\}$$

CFG for $a^n b^m$ where $n \geq m$, & $m \geq 1$

$$S \rightarrow aAb \mid asb$$
$$A \rightarrow aA|a$$

cf G for $a^n b^m$ where $n > m$ & $m \geq 2$

$$S \rightarrow aAb \mid aSb$$
$$A \rightarrow aA'b$$
$$A' \rightarrow aA' \mid a$$

$S \rightarrow aaAbb$ (1) ~~aab~~

$$A \rightarrow aAb \mid aA'b$$
$$A' \rightarrow \cancel{A} \cap A' \mid a$$

Done Neatly :D

Production

$$S \rightarrow aaAbb \mid aaA'bb$$
$$A \rightarrow aAb \mid aA'b$$
$$A' \rightarrow aA' \mid a$$

→ CFG

Date: _____

$$L = \{ a^i b^j c^k d^l \mid i+j = k+l \}$$

$i, j, k, l \geq 0$

Possibilities:

① bc ② ad ③ bd

abdd

abcc

aa dd

bbcd

④ aacc

~~$S \rightarrow aAc$~~

outer most

$S \rightarrow a S d \mid A \mid B \mid C$

$A \rightarrow a A c \mid C$

$B \rightarrow b B d \mid C$

$C \rightarrow b C c \mid \epsilon$

~~$i=j$~~ $L = \{ a^i b^j c^k d^l \mid i+j \geq k+l \}$

$\& i, j, k, l \geq 0$

now $\geq$ instead of $=$

$S \rightarrow a S d \mid A \mid B \mid C \mid A_1 B_1$

$A \rightarrow a A c \mid C \mid A_1 B_1$

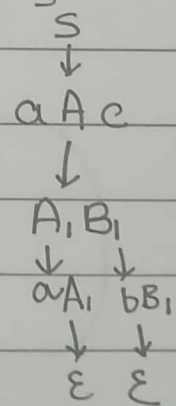
$B \rightarrow b B d \mid C \mid B_1$

$C \rightarrow b C c \mid \epsilon \mid B_1$

$A_1 = a A_1 \mid \epsilon$

$B_1 = b B_1 \mid \epsilon$

$\rightarrow aabc$



$$L = \{ 0^i 1^j 2^k \mid i=j \text{ or } j=k, i, j, k \geq 0 \}$$

$$S \rightarrow \cancel{0A1T_1} \mid 0_1 1 B 2$$

$$A \rightarrow 0 A_1 1 \mid \epsilon$$

$$B \rightarrow 1 B 2 \mid \epsilon$$

$$T_1 \Rightarrow 2 T_1 \mid \epsilon$$

$$0_1 \rightarrow 0 0_1 \mid \epsilon$$

012

0112001201220011201122 $i=j:$ $j=k:$

$$S_1 \Rightarrow A_1 B_1$$

$$S_2 \rightarrow A_2 B_2$$

$$A_1 \rightarrow 0 A_1 1 \mid \epsilon$$

$$A_2 \rightarrow A_2 0 \mid \epsilon$$

$$B_1 \rightarrow B_1 2 \mid \epsilon$$

$$B_2 \rightarrow 1 B_2 2 \mid \epsilon$$

What if : $i=j$ OR $j=k$ --- then:

 $i=k:$

$$S_3 \rightarrow 0 S_3 2 \mid A_3$$

$$A_3 \rightarrow A_3 1 \mid \epsilon$$

0011 22

↳ is being generated by all three parts

||| Ambiguity |||

when multiple productions can create the same string.

$$L = \{ \underbrace{a^n b^p c^m d^2}_{\text{balance these pairs}} \mid n=L \ \& \ p=m \}$$

$$S \rightarrow a S d \mid A$$

$$A \rightarrow b A c \mid \epsilon$$

Simplification } Ambiguity

Date: _____



Wherever there's any confusion.

Say:

$$S \rightarrow (S) \mid \{s\} \mid [s] \mid SS \mid \epsilon$$

String: $()()()$

Production can be tracked easily: (There's no confusion here)

→ (left-most derivation)

$$\begin{aligned}
 S &\rightarrow \underline{SS} \\
 &\quad \downarrow \\
 S &\rightarrow \underline{S} S S \\
 &\quad \downarrow \\
 S &\rightarrow (S) SS \\
 &\quad \downarrow \\
 S &\rightarrow (\epsilon) SS \\
 &\quad \downarrow \\
 S &\rightarrow () SS \\
 &\quad \downarrow \\
 S &\rightarrow () (S) S \rightarrow () (\epsilon) S \rightarrow () () S \rightarrow () () (\epsilon) \\
 &\quad \downarrow \\
 S &\rightarrow () () ()
 \end{aligned}$$

// No ambiguity.
 // This derivation is unique
 // will always be the same for
 $()()()$
 OR WILL IT???
 $()(\epsilon)S \rightarrow ()()S \rightarrow ()()(\epsilon)$

~~Now!! Let's modify~~ How?

Ambiguity

Sooo... when there's more than 1:

- left derivation
- right derivation
- parse tree

$$S \rightarrow SS$$

$$\rightarrow \epsilon S$$

$$\rightarrow \epsilon \underline{S} S$$

$$\rightarrow \epsilon S \underline{S} S$$

$$\vdots$$

$$\rightarrow ()()()$$

→ Another derivation that can lead to $()()()$

Then, the grammar is ambiguous.

Simplification:

Date: _____

Two Major Stuff:

- ① Clean the grammar $\rightarrow$ remove the stuff that's creating garbage!
- ② Simplify the grammar.

Consider the following Grammar:

$S \rightarrow AB \mid BC \mid CD \mid F$

$A \rightarrow aala$

$B \rightarrow bb \mid b$

$D \rightarrow \epsilon \mid D$

$E \rightarrow ae \mid ABD$

What's wrong here?

C & F have no productions

E is not reachable.

D is not terminating.

let's say D is now terminating at 'd'

Update Grammar:

$S \rightarrow AB \mid \cancel{BC} \mid \cancel{CD} \mid \cancel{F}$

$A \rightarrow aala$

$B \rightarrow bb \mid b$

$\cancel{D \rightarrow d}$ // assumption from above.

$\cancel{E \rightarrow ae \mid ABD}$

$\rightarrow$ since C has no productions, any production it's a part of becomes invalid.

b/c no productions.

$\rightarrow$ since D got removed with C, then D is now unreachable too.

$\rightarrow$ since you cannot reach E from S, you can eliminate it.

Order of removal:

- ① Non-Terminating
- ② Non-Reachable

Cleaning is Done, Now onto Simplification.

$\Rightarrow$ Simplification:

- ϵ , Null Production Removal
- Unit Production Removal

Example for Simplification:

$$S \rightarrow AB \mid A \mid BA \mid AA$$

$$A \rightarrow aAb \mid aAc$$

$$B \rightarrow bbA \mid bbC \mid \epsilon$$

$$C \rightarrow cC \mid \epsilon$$

• When removing ϵ -Productions: You cannot remove ϵ from ~~area~~ when the ϵ is a part of the language that the grammar is for.

So, what can we do?

Let's say $G(L)$ is a grammar for a language with ϵ . So we take G' , which is a grammar for a language L' which doesn't contain ϵ .

$$L' = L - \epsilon$$

$$\& L' \cup \epsilon = L$$

$$\text{then } G' \cup \epsilon = G.$$

Now go back & look at the grammar above & see where the ϵ -productions are.

Take this production: $C \rightarrow cC \mid \epsilon$

if we consider cC & replace C with ϵ then this generate $c\epsilon$ or c .

Replace ϵ with c then.

New production: $C \rightarrow cC \mid c$

$B \rightarrow bbA \mid bbC \mid \epsilon$ also has C, & C led to ϵ
now what?

was: $bbC \rightarrow bb \underline{cC} \mid bb \underline{\epsilon}$

so bb would have been a production that we can't reach anymore... so... let's add 'bb' as a production now.

$B \rightarrow bbA \mid \underline{bbC} \mid \epsilon \mid bb$

↳ never replace this.

Now for $A \rightarrow aAb \mid aAC$

Same logic as above:

$A \rightarrow aAb \mid aAC \mid aA$

Now remove ϵ -Production from B

$B \rightarrow bbA \mid bbC \mid (\epsilon) \mid bb$

↳ remove it.

Update $A \rightarrow aAb \mid aAC \mid aA$ // doesn't have B productions
no changes.

Update $S \rightarrow \underline{AB} \mid A \mid \underline{BA} \mid AA$

↳ remove B (since ϵ) results in A. A is already a production so cannot be added again.

If $S \rightarrow \underline{\quad}$ & had a BB then...

ϵB or $\epsilon \epsilon$ then we could add $B \mid \epsilon$

$S \rightarrow AB \mid A \mid BA \mid AA \mid B \mid \epsilon$

Then ...

$$G' \Rightarrow S \rightarrow (S' | \epsilon$$

& $S \rightarrow$ // rest of what we did.

rest of the stuff

Date: _____

Is that ϵ from the end.

UNIT PRODUCTIONS:

A variable without any other variable or terminal

$$A \rightarrow \underline{B} | \dots$$

$$B \rightarrow \underline{C} | \dots$$

$$C \rightarrow \underline{D} | \dots$$

$$D \rightarrow \underline{\epsilon} | \dots$$

(not ϵ , Epsilon.)

} are on the right hand side.
unit productions.

$$A \rightarrow B$$

$$A \rightarrow (JK)$$

$$A \rightarrow K$$

You just replace the unit production with all of its productions.

From the previous example:

$$S' \rightarrow AB | \underbrace{aAb | aAc | aA}_A | BA | AA | BB | \underbrace{bBa | bBc | bB}_B$$

$$A \rightarrow aAb | aAc | aA$$

$$B \rightarrow bBa | bBc | bB$$

$$c \rightarrow cC | c$$

Order for Removal:

- Remove ϵ -Productions
- Remove Unit Productions.

Why 2 variables?

- Binary Tree!! Good for programming.

Chomsky Normal Form (CNF)

Date: _____

- o Pair of variables
 - o ^{one} terminal ~~X~~
- ✓ → $AA \mid a$

Continue with previous Grammars

✓ = in format
X = not in format

$S \rightarrow \checkmark AB \mid \checkmark BA \mid \checkmark AA \mid \checkmark BB \mid \times aAb \mid \times aAC \mid \times aA \mid \times bbA \mid \times bbC \mid \times bb \mid \checkmark \epsilon$

We know what the terminals are... (not counting ϵ)
So we know what the alphabet set is.

$$\Sigma_1 = \{a, b, c\}$$

STEP #1:

So add some Productions for Terminals:

$$T_1 \rightarrow a$$

$$T_2 \rightarrow b$$

$$T_3 \rightarrow c$$

$$A \rightarrow \times aAb \mid \times aAC \mid \times aA \rightarrow A \rightarrow V_1 T_2 \mid V_1 C \mid T_1 A$$

$$B \rightarrow \times bbA \mid \times bbC \mid \times bb \rightarrow B \rightarrow V_2 A \mid V_2 C \mid T_2 T_2$$

$$C \rightarrow \times cC \mid \times c \rightarrow C \rightarrow T_3 C \mid c$$

Step #2:

Replace terminals with $T_1, T_2, T_3 \dots$

aAb

becomes
 $T_1 A T_2$

bbC

becomes

$T_2 T_2 C$

aAC

$T_1 AC$

bb

$T_2 T_2$

are now in format

aA

$T_1 A$

~~bbA~~

bbA

$T_2 T_2 A$

cC

$T_3 C$

Make more productions:

$$V_1 \rightarrow T_1 A$$

// now replace $T_1 A$ s with V_1

$$V_2 \rightarrow T_1 T_2$$

// " " $T_1 T_2$ with V_2

$T_1 A T_2$ becomes
 $T_1 A C$

$V_1 T_2$
 $V_1 C$

$\{ T_2 T_2 A$
 $T_2 T_2 C$

becomes
"

$V_2 A$
 $V_2 C$

PDA

Date: _____

Push Down Automata $\rightarrow$ Stack + ϵ -NFA.

$a^n b^n$

$$S \rightarrow a S b \mid \epsilon$$

$\hookrightarrow$ since ϵ -NFA, therefore also non-deterministic.

word: $\rightarrow$ a a a a b b b b

Transition Functions so far:

DFA $\delta(Q, \epsilon) = Q$

NFA $\delta(Q, \epsilon) = \{q_1, \dots, q_n\}$

ϵ -NFA $\delta(Q, \epsilon \cup \epsilon) = \{ \text{states you reach via } \epsilon\text{-closure} \}$.

Transition Function for PDA:

$$\delta(Q, \epsilon \cup \epsilon, x) \rightarrow \{ (Q_1, Y_1), (Q_2, Y_2), \dots \}$$

$\downarrow$
current top of the stack.

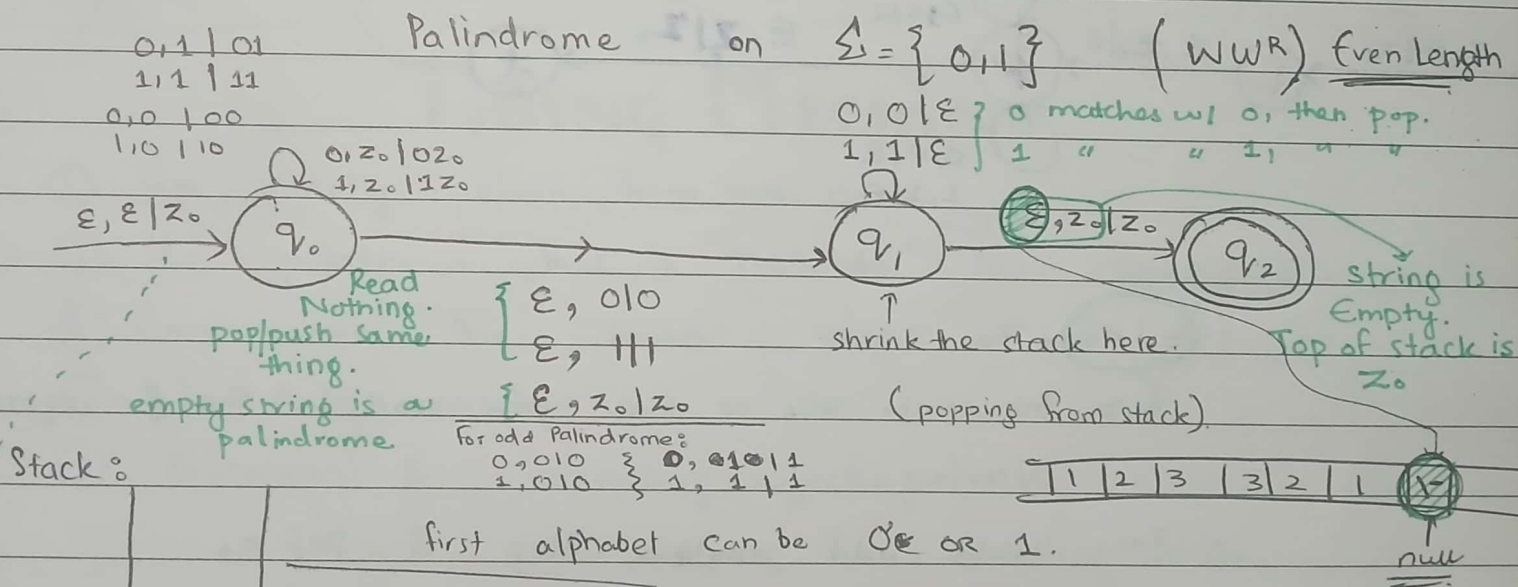
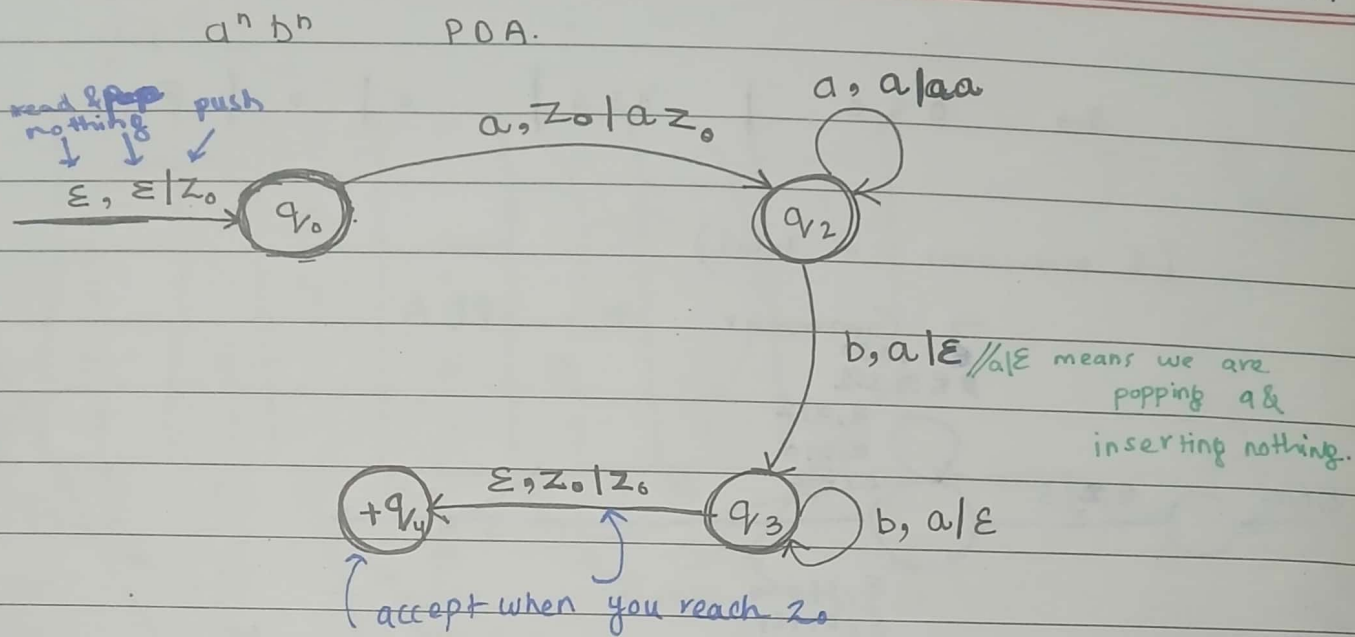
Y is gonna be symbols that can be inserted into the stacks.

Symbols for stack are independant of the alphabet set Σ .

- Y can include:
- $Y = \epsilon$ // ϵ // pop from stack if on left
 - $Y = x$ // single character
 - $Y = z_1 z_2 z_3 \dots z_n$ // Word.

$\hookrightarrow$ is pushed alphabet by alphabet into stack.

Starting from z_n to z_1



Stack :

first alphabet can be 0 or 1.

• We can use a constant to mark the center of a string.

• constant cannot be part of the string.

WWR

$$\begin{array}{c} 011 \\ \hline w \\ \hline z_0 \end{array} \quad \begin{array}{c} 2 \\ \hline c \\ \hline \epsilon \end{array} \quad \begin{array}{c} 110 \\ \hline w \\ \hline \epsilon \end{array}$$

second step,

$$\begin{array}{l} 0, 0100 \\ 1, 0110 \end{array} \quad \begin{array}{l} 0101 \\ 1111 \end{array}$$

0, z_0 | 0 z_0

1, z_0 | 1 z_0

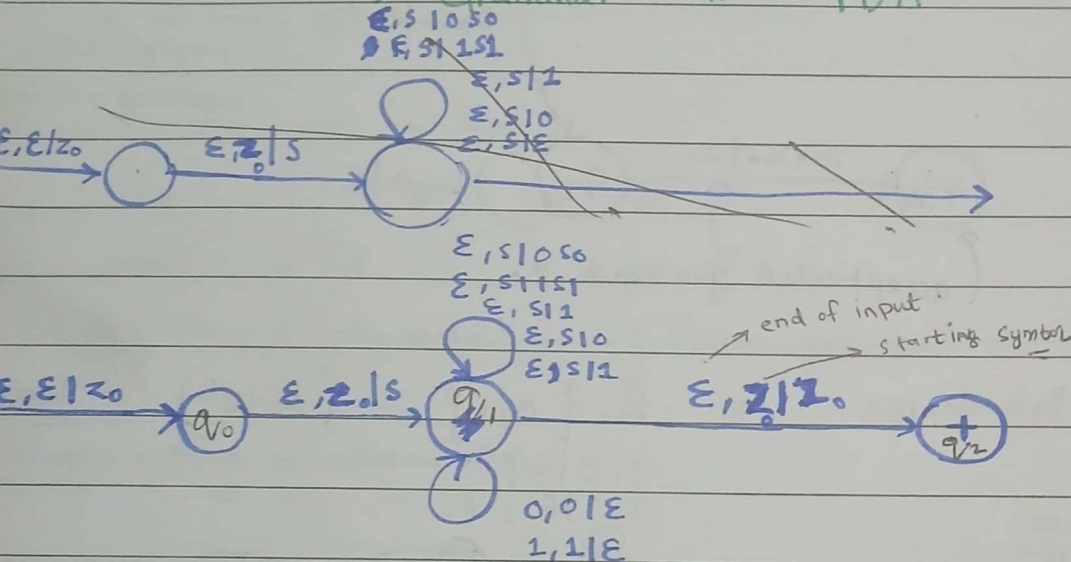
pop push

Stack will be $\begin{bmatrix} 1 \\ z_0 \end{bmatrix}$

$S \rightarrow 0S0 \mid 1S1 \mid 1 \mid 0 \mid \epsilon$

(5 productions in total)

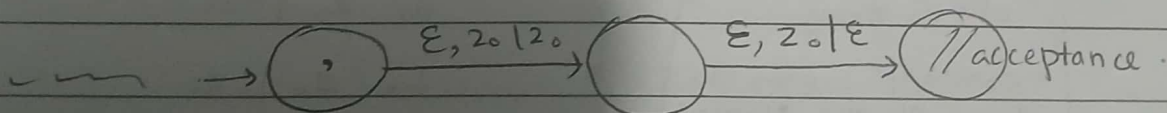
→ Grammar to PDA

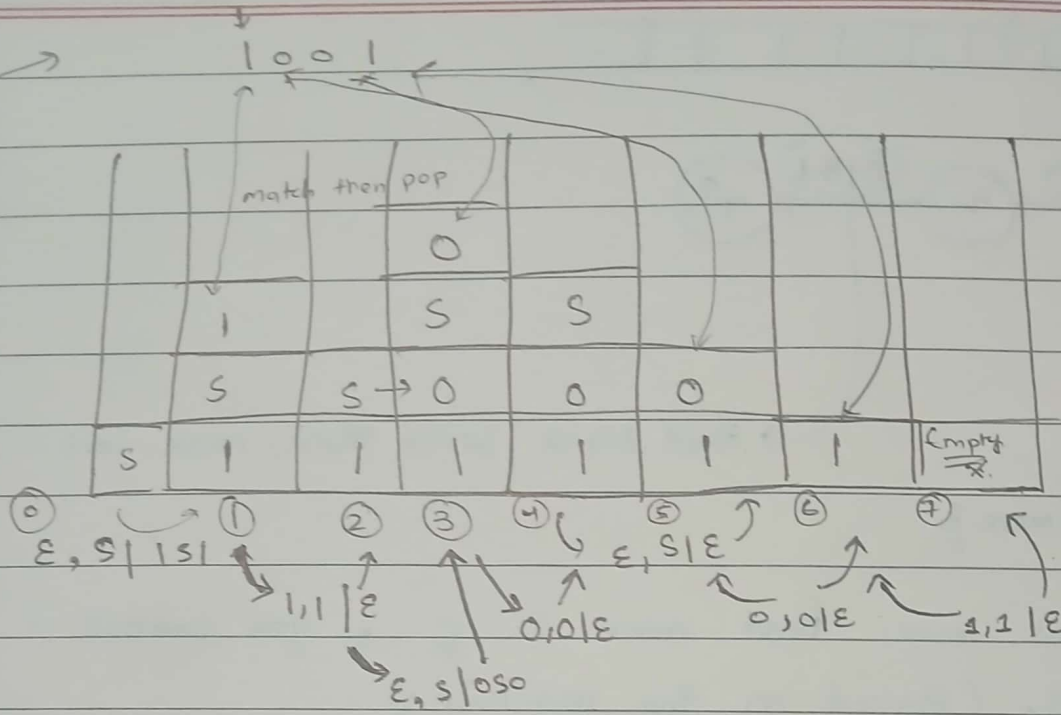


$\epsilon | z_0, z_0$

→ this combo tells us that you've reached the acceptance criteria.

- you could add another step & pop z_0 .
- when stack is empty, then that is also acceptance.

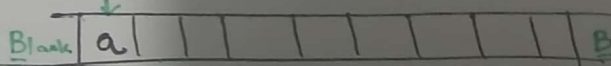


April 30th 2025

TURING MACHINE

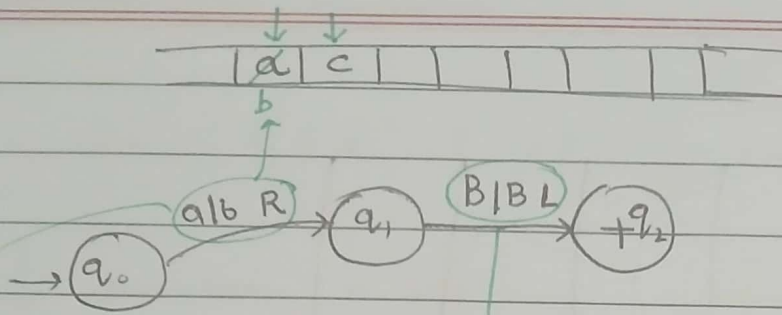
- ① Tape initially blank from both sides.
- ② No pointer, but a "Header". Then read/write operations.
- ③ Can move the 'head' right or left $\rightarrow, \leftarrow$
- ④ Is deterministic.
- ⑤ Halting & accepting/rejecting.
 - $\rightarrow$ No outgoing from accepting state.
 - $\rightarrow$ No loops, etc.
 - $\rightarrow$ Wherever the TM stops it's called "halting" state.

Head: left-most non-blank cell. (in this case, a)



B = Blank here.

- Steps:
- 1- Read a
 - 2- write b
 - 3- Move R/L.



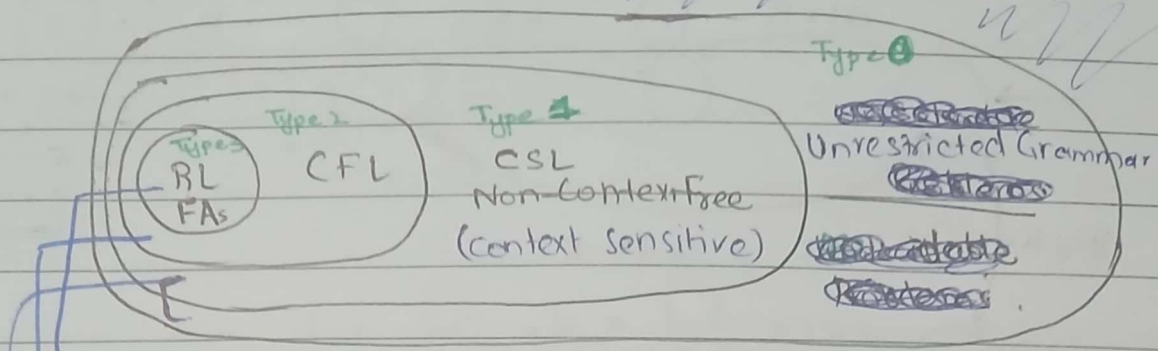
→ Read blank, write Blank, move Left.

→ Read a, write b, move R.

→ after first move, head moves to c, you cannot proceed. (Based on the machine.)

→ You're now in the halting state.

✓✓✓✓✓ Unsolvable problems



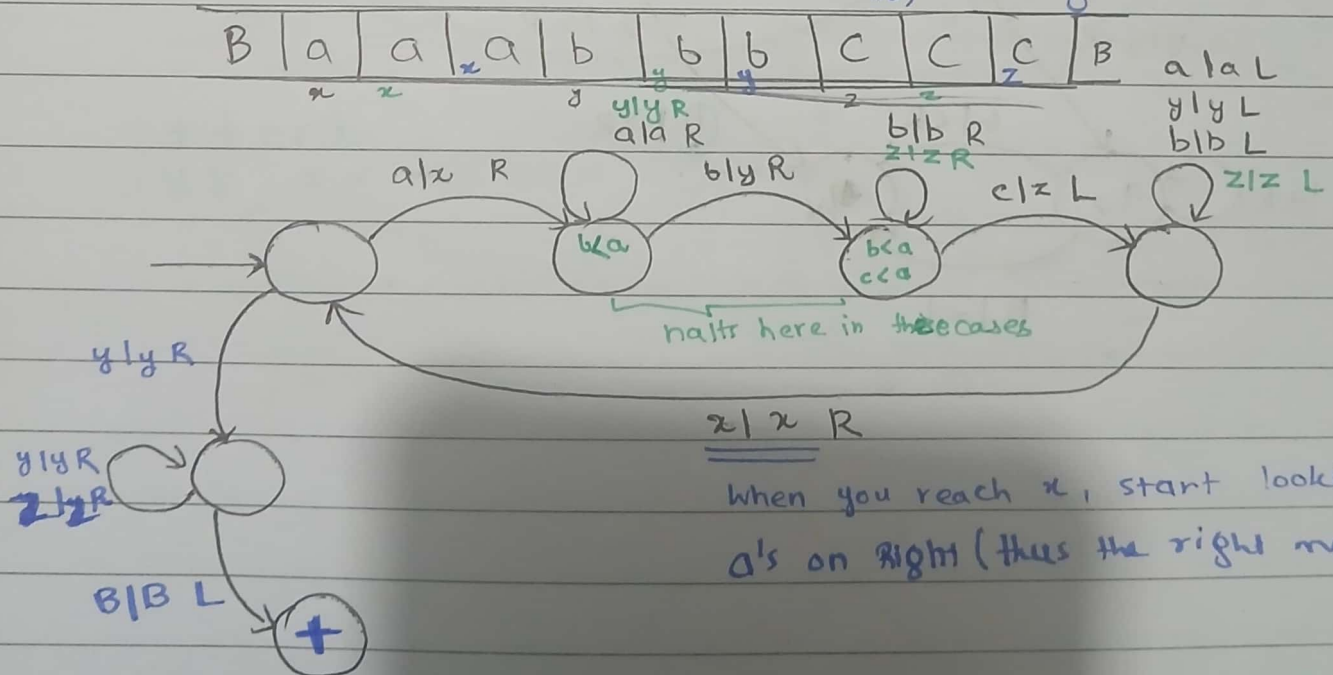
→ Regular Languages / finite Automata / Regular Expressions.

→ Context Free Languages (PDAs).

→ Turing machine (can solve any problem that is not undecidable).

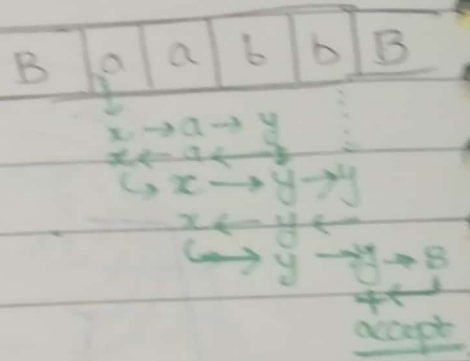
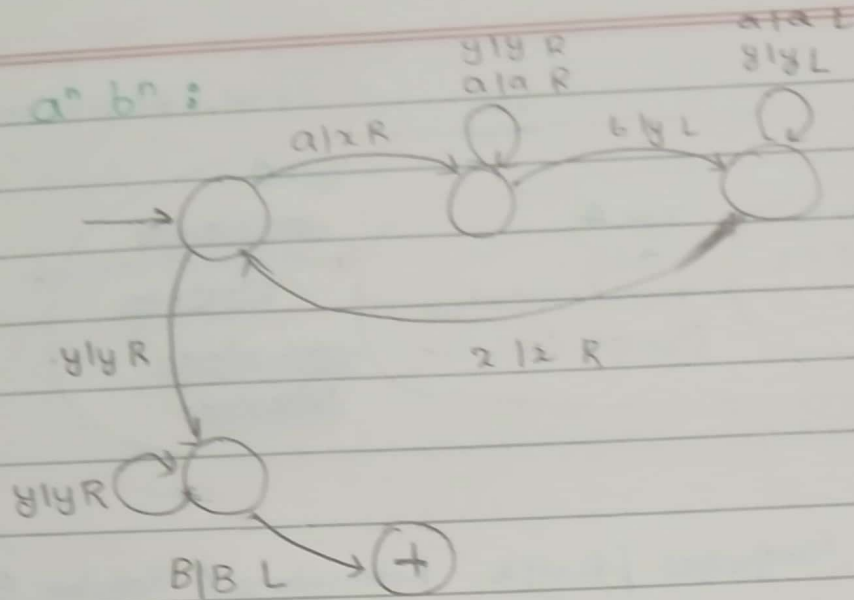
Solving $a^n b^n c^n$ with TMs.

Keep in mind TM is deterministic, so only one transition per character

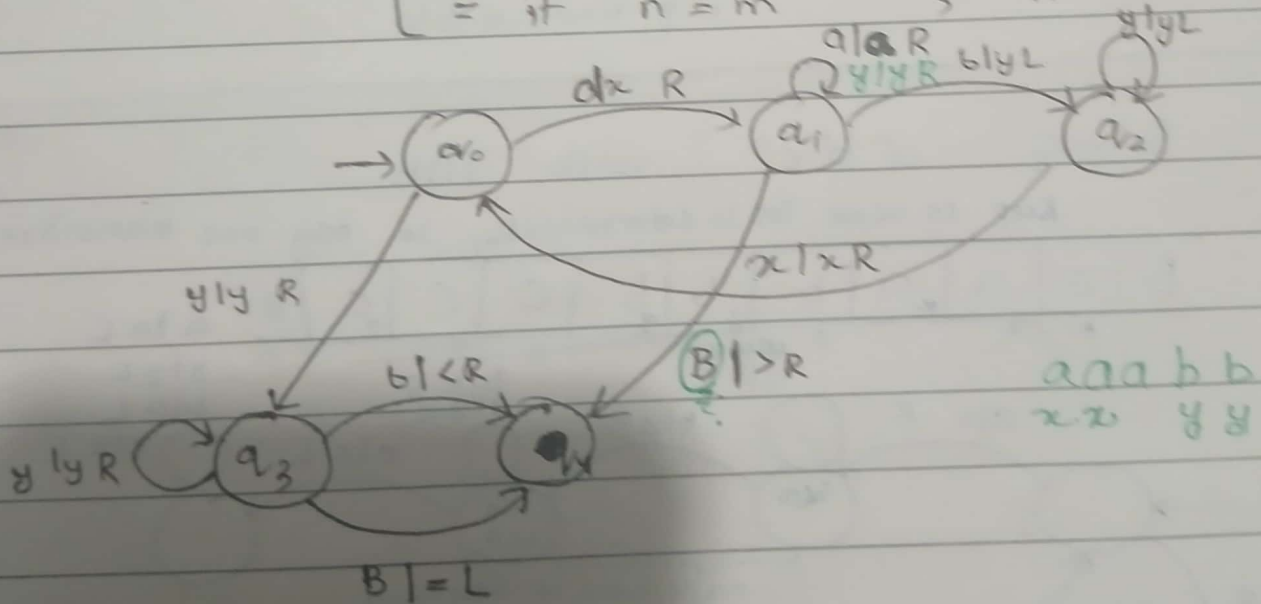


How will you convert this to $a^n b^n$?

TM for $a^n b^n$:



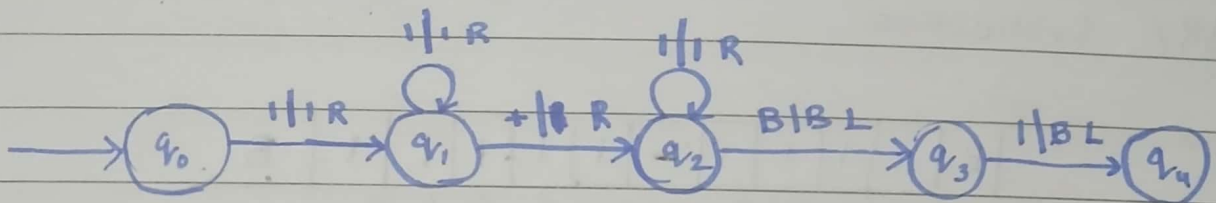
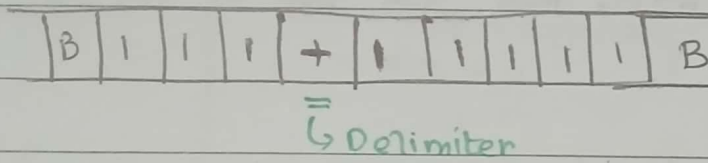
$$f(n, m) = \begin{cases} > & \text{if } n > m \\ < & \text{if } n < m \\ = & \text{if } n = m \end{cases} \quad , \quad \begin{matrix} a > b \\ a < b \\ n = m \end{matrix}$$



— UNARY ADDITION

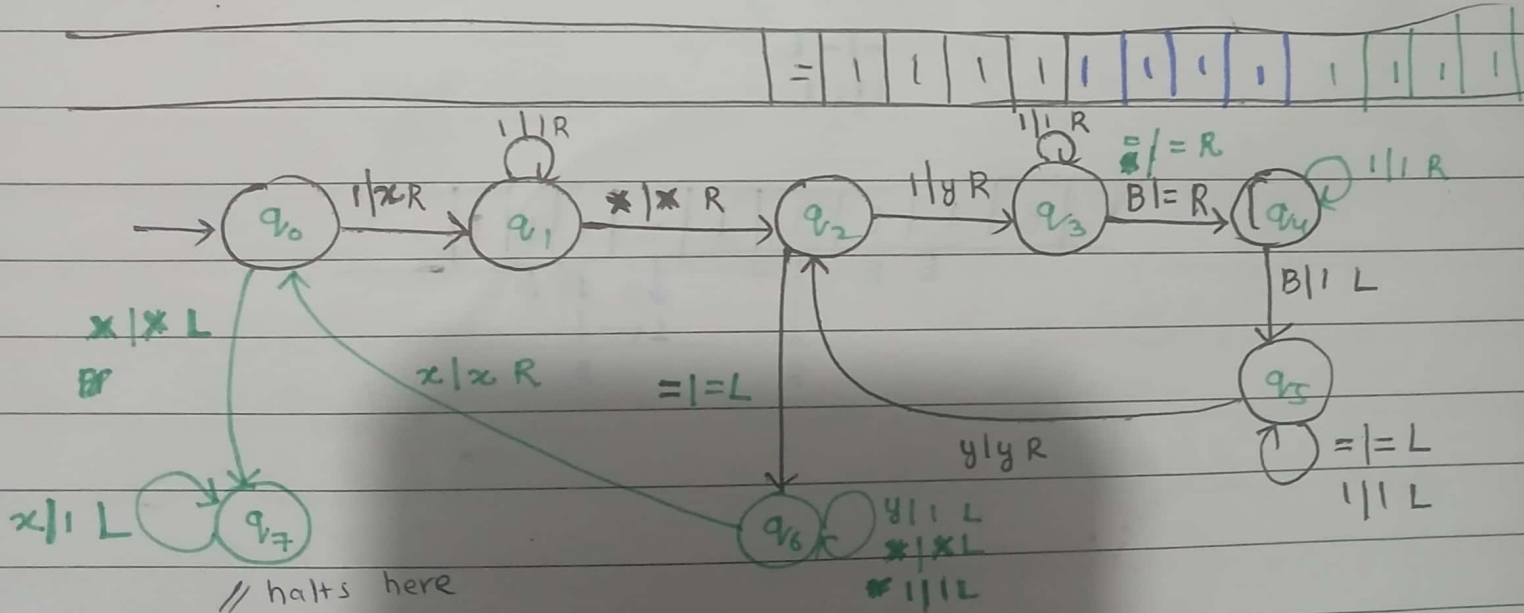
Date: _____

$\Sigma = \{ 1 \}$ // unary system.



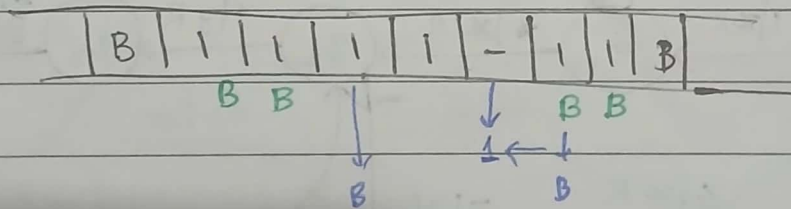
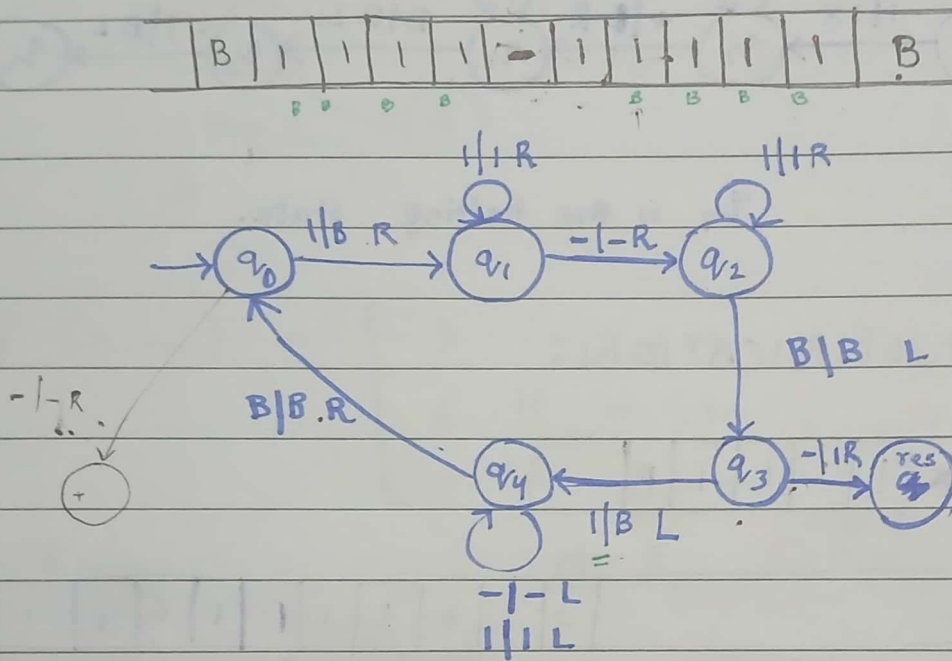
q_4 is the halting state.

— UNARY MULTIPLICATION:



Minus = $a - b$, if $a < b$, then value would be -ve.
 "Minus" = $a - b$, if $a < b$, then you stop at 0, not -ve.

UNARY Subtraction.

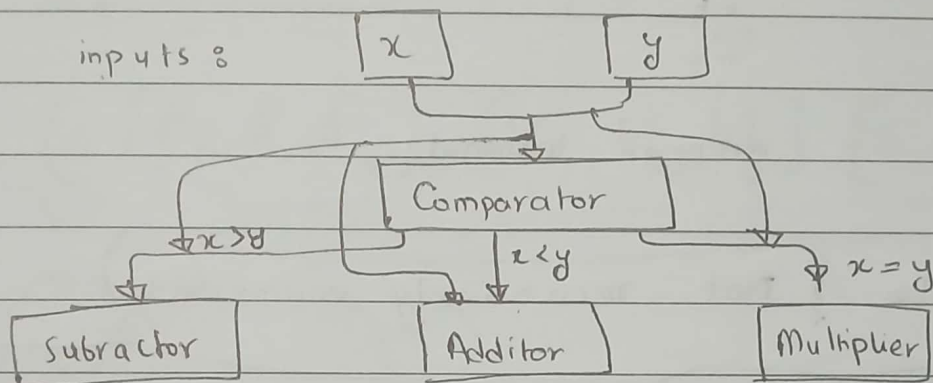


Combining Turing Machines:

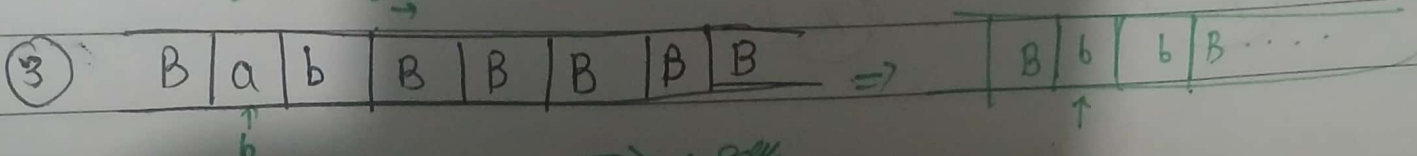
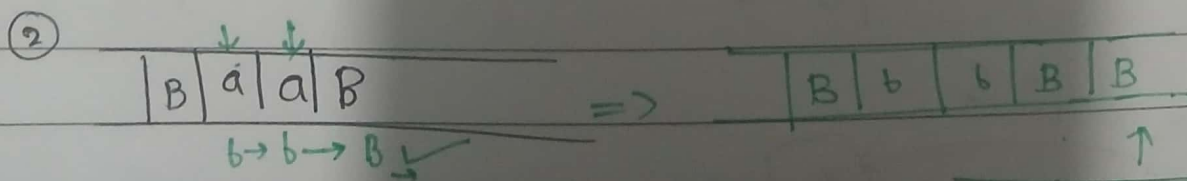
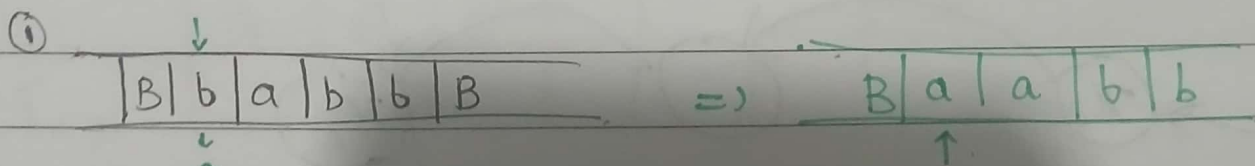
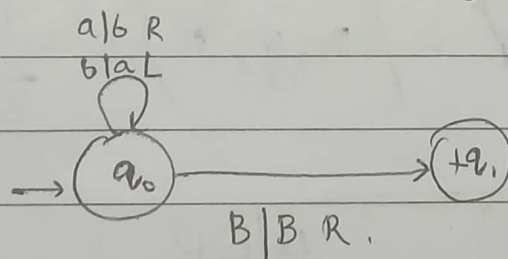
Date: _____

$f(x,y) = \left\{ \begin{array}{l} x + y, \text{ if } x < y \\ x - y, \text{ if } x > y \\ x * y, \text{ if } x = y \end{array} \right.$	$Tm1 = \text{Additor}$	$Tm4 = \text{Comparator}$
	$x + y, \text{ if } x < y$	$x < y$
	$x - y, \text{ if } x > y$	$x > y$
	$x * y, \text{ if } x = y$	$x = y$

First make all 4 TMs then solve the function on (x,y)

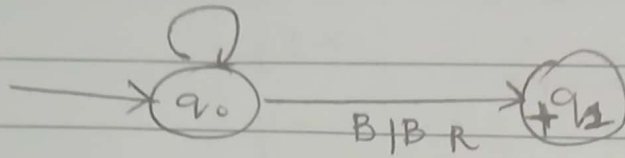


Non-Halting TM



$a \mid aR$

$b \mid bL$



3rd one

B | a b | B | B | B | B | B

$a \rightarrow$
 $\leftarrow b$
 $a \rightarrow$
 $\leftarrow b$
 $\vdots$

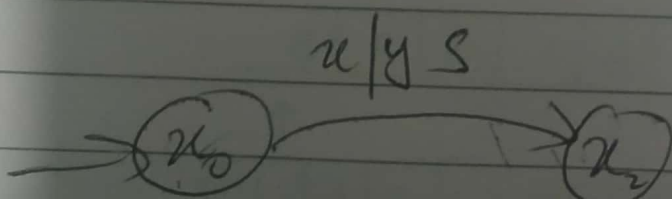
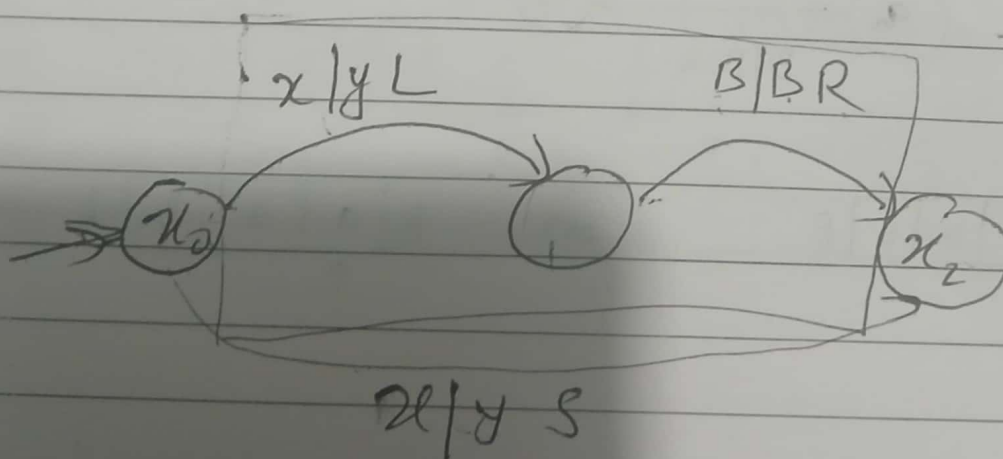
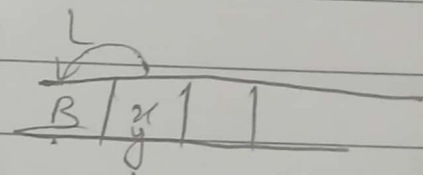
Will never terminate.

(Undecidable } Not recursively enumerable)

VARIANTS OF TURING MACHINES

Stay option

$$\Sigma = \{x, y, B\}$$



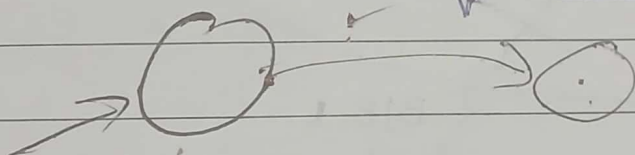
Multitape

T₁ B | 1 | 1 | 1 | B | 1 | 1 | 1

T₂ B | 1 | 1 | 1 | 1 | 1 | (B) | 1 | 1 | 1

T₃ B | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1

$([R_1, R_2, R_3], [W_1, W_2, W_3], [D_1, D_2, D_3])$



Multi track

	a	b	c			
T ₁	A	B	C			
T ₂						
T ₃						

$\{(R_1, R_2, R_3), (W_1, W_2, W_3), D\}$

~~COG 10~~
Enzym

W

3.0.00
3.0.00
3.0.00

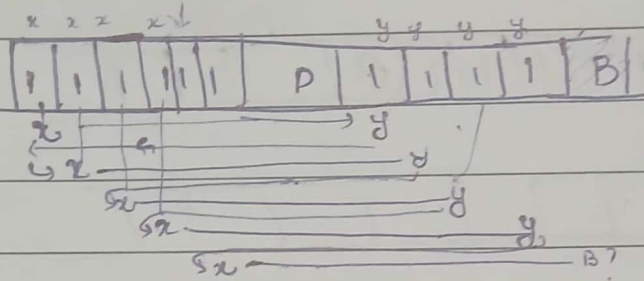
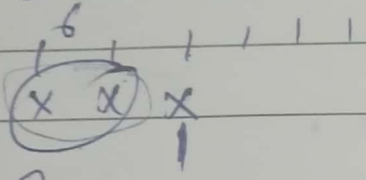
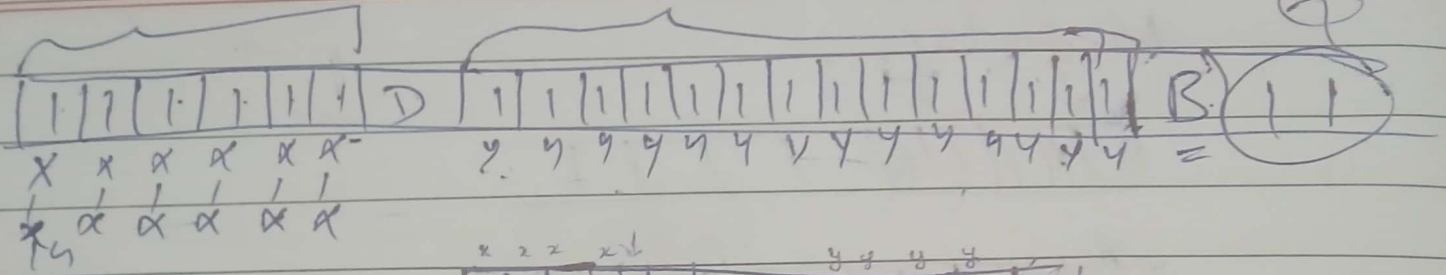
1

14

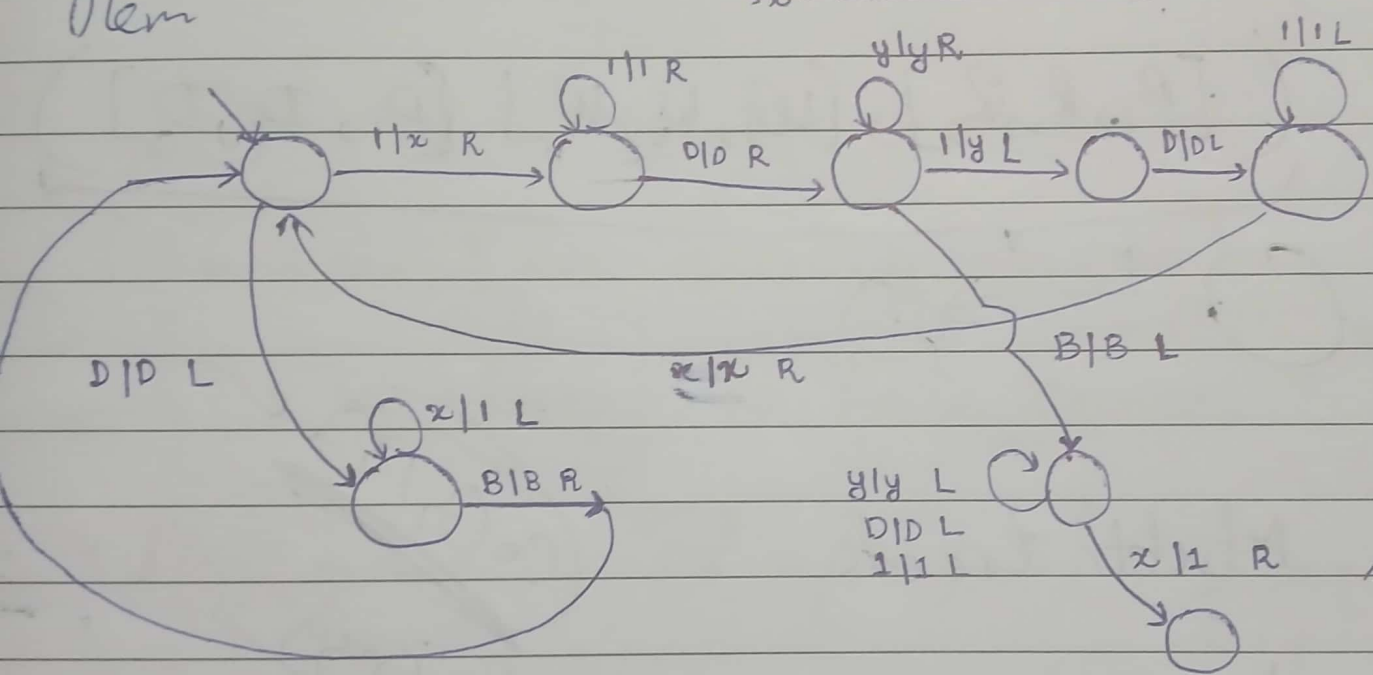
Date: _____

a

b

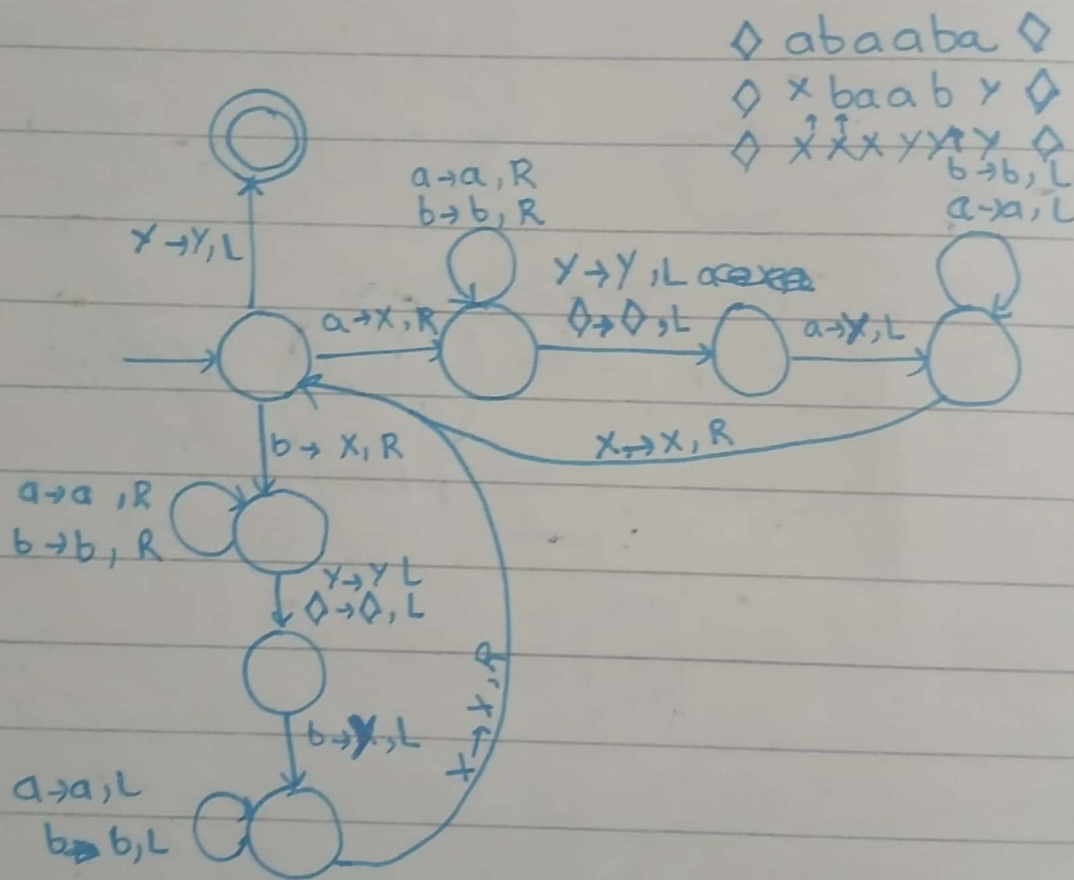


Rem

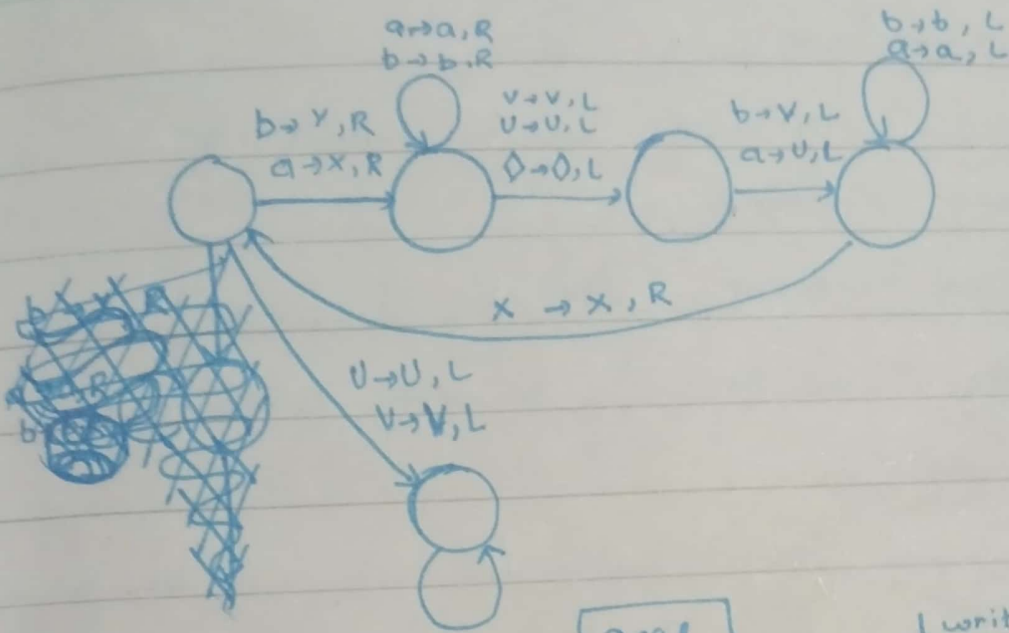


// no. of x is the remainder

$L_1 = \{ v v^R : v \in \{a, b\}^* \}$
 $L_2 = \{ v v : v \in \{a, b\}^* \}$



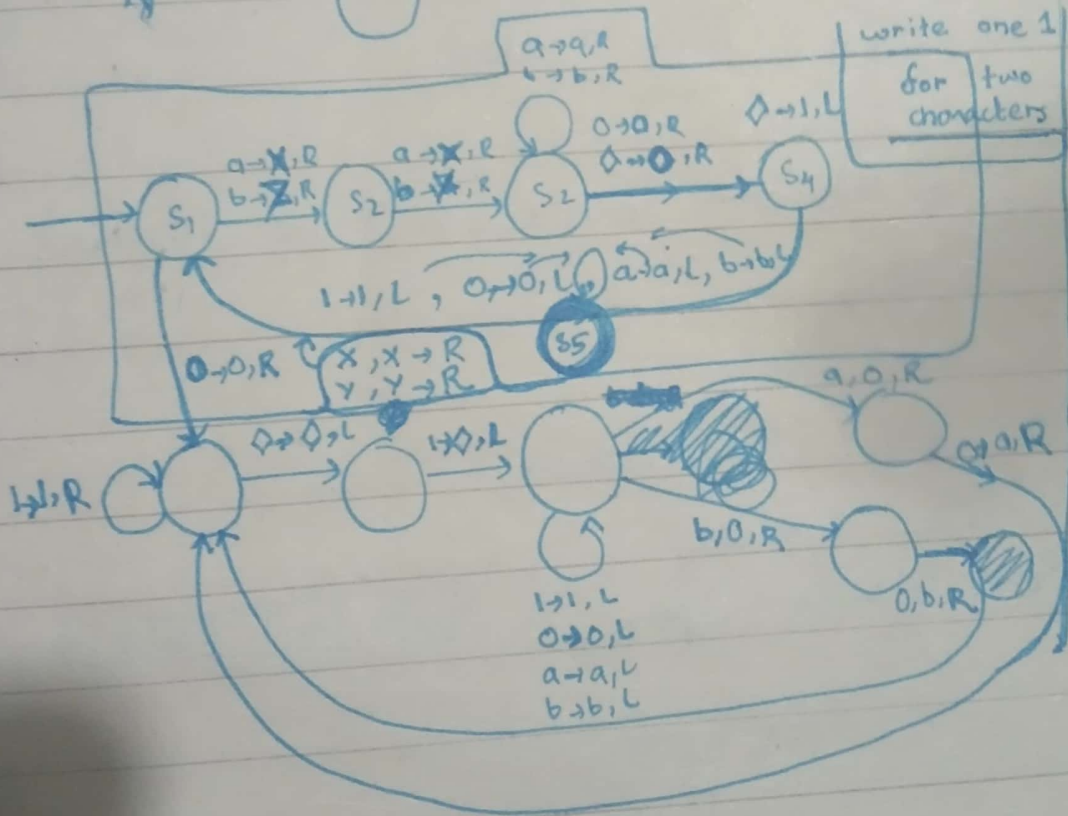
$XY \{ UV$



~~babbb~~

~~babbb~~

ababa



9. 1001 2 9, 1010,

$$E = \{0, 1\}^*$$

Binary Increment.

000001

0010 → 0011

1100 → 1107.

DOI 10.101

1010010

10001

9 101000

①

→

1010100

0 1 0 1 0 1 0

1

1

Diagram illustrating a transition from state q_0 to state q_1 labeled $\Delta \rightarrow 1, L$.

②

 $x \rightarrow 0, R$

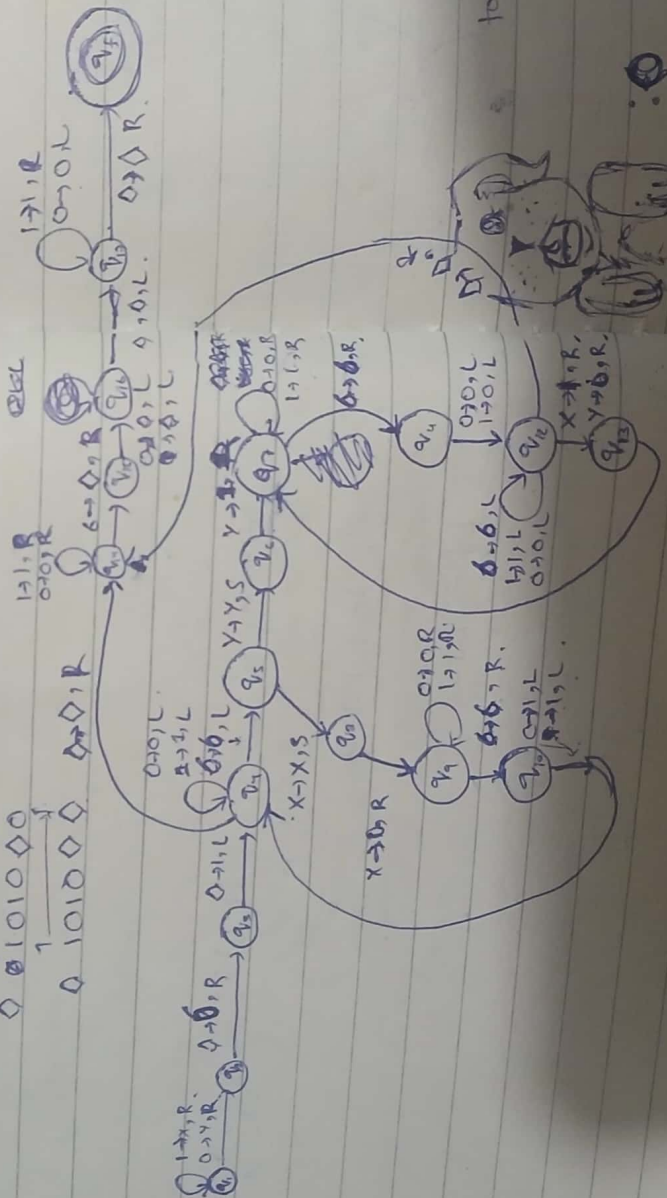
1

10

1

This image shows a blank sheet of white paper with faint, horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page. There is no handwriting or other markings on the paper.

2

[illegible]

Handwritten binary addition of 10010101 and 10101010. The calculation is shown in two columns, with the result 10101111 written below. The carry is indicated by a '1' at the top right.

1	0	0	1	0	1	0	1
1	0	1	0	1	0	1	0
1	0	1	0	1	1	1	1

0010100

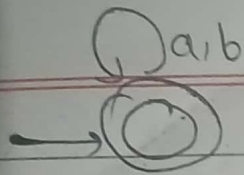
Addition only have

to use carry till you reach 0.

1011111
1100000

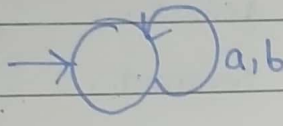
A hand-drawn diagram of a simple neural network. It consists of two input nodes on the left, each represented by a circle with a vertical line extending from its center. These input nodes are connected to a single output node on the right, which is represented by a circle with a vertical line extending from its center. The connections are shown as horizontal lines between the nodes.

Date: 12/2/2024



$$\rightarrow [(a+b)^*]^c \Rightarrow \emptyset$$

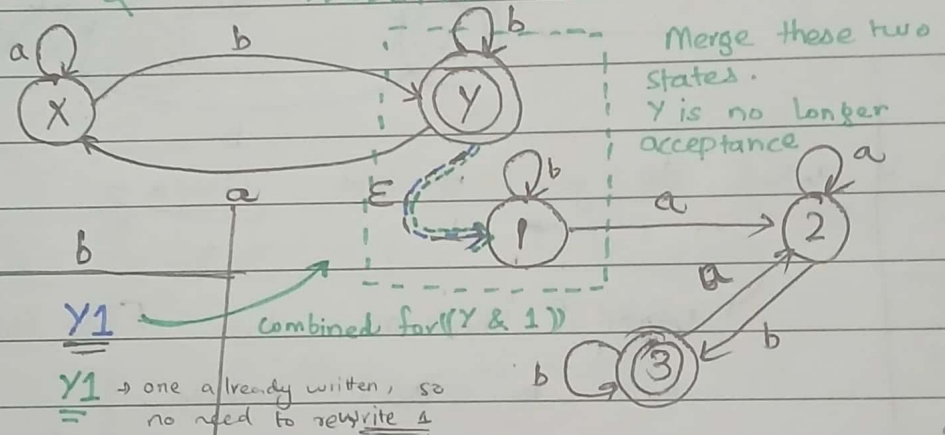
Compliment



accepts nothing.

ϵ -NFA to DFA $\rightarrow$ Next Monday.

③ CONCATENATION & KLEENE STAR



initial state $\rightarrow$ X

Y1

X2

* Y13

final state

a

X

X2

X2

on 3 a goes to 2, 2 is already there.

b

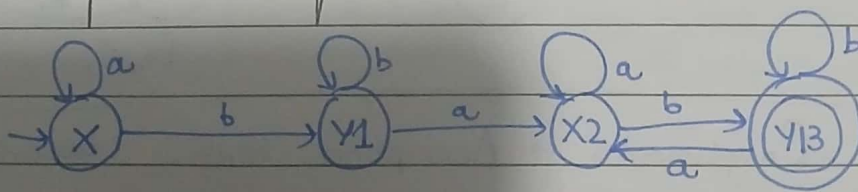
Y1

Y1

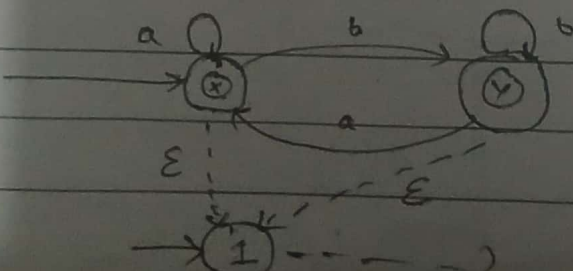
Y13

Y13

$\rightarrow$ one a already written, so no need to rewrite a



What to do if there's multiple final states on the first machine?



X1 $\rightarrow$ would be one state

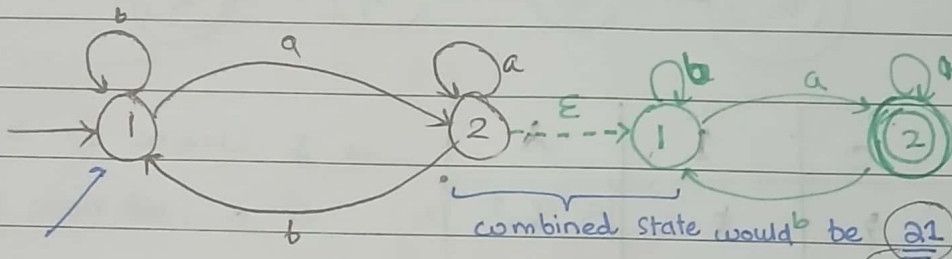
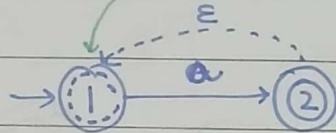
Y1 $\rightarrow$ would be other state.

→ KLEENE STAR

language $\Rightarrow A$

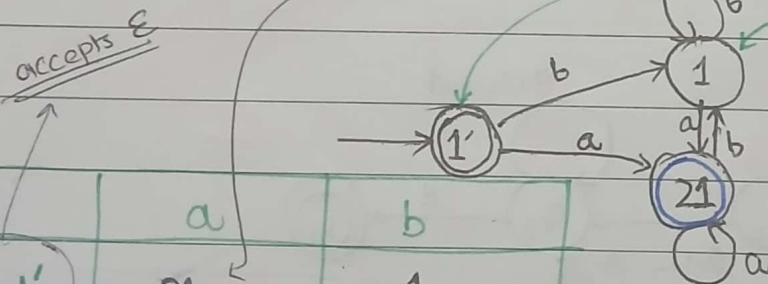
Kleene Star of $A \Rightarrow (A)^* = \{\epsilon, A, AA, AAA, \dots\}$

→ To accept ϵ , make initial state to be an acceptance state ←



→ to make it an acceptance state for ϵ , but it also has incoming edges (like b , here) which is not accepted by the language/DFA. (b is not accepted by 1) (1 prime)

→ We need to make a clone of 1 → Accepting 1 (or 1')
→ Non Accepting 1.



	a	b
21	21	1
21	21	1
1	21	1

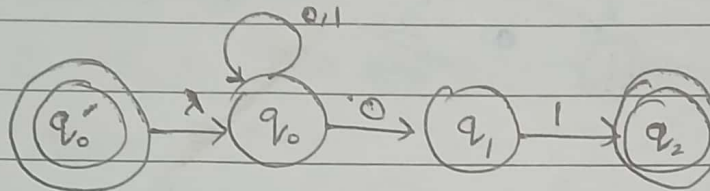
You need to not split when:

- ① Initial is already an acceptance state
- ② Initial is a non-acceptance state with incoming edges.

ϵ -NFA or Λ -NFA or λ -NFA

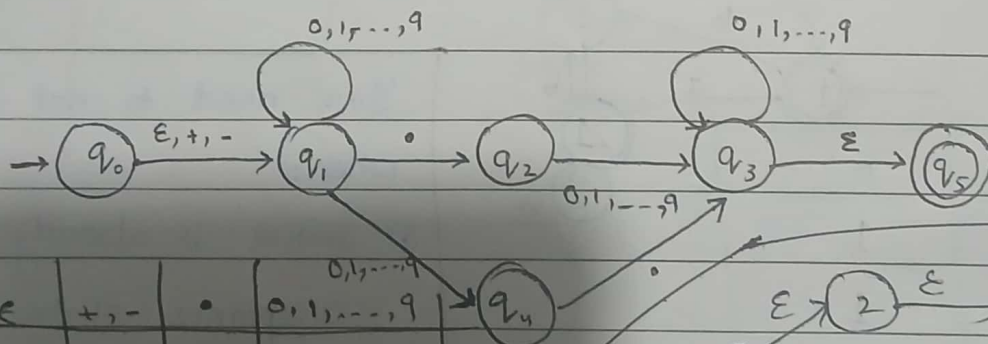
o) λ -NFAs are those NFAs with at least one explicit λ -transition defined.

Example:



δ	0	1	λ	$\rightarrow \epsilon$ -Closure or λ -Closure of a node: nodes that it can reach by λ -transitions.
$\rightarrow * q_0'$	ϕ	ϕ	$\{q_0', q_0\}$	
q_0	$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0\}$	$C(q_0') = \{q_0', q_0\}$
q_1	ϕ	$\{q_2\}$	$\{q_1\}$	
$* q_2$	ϕ	ϕ	$\{q_2\}$	

ϵ -Closure Example:



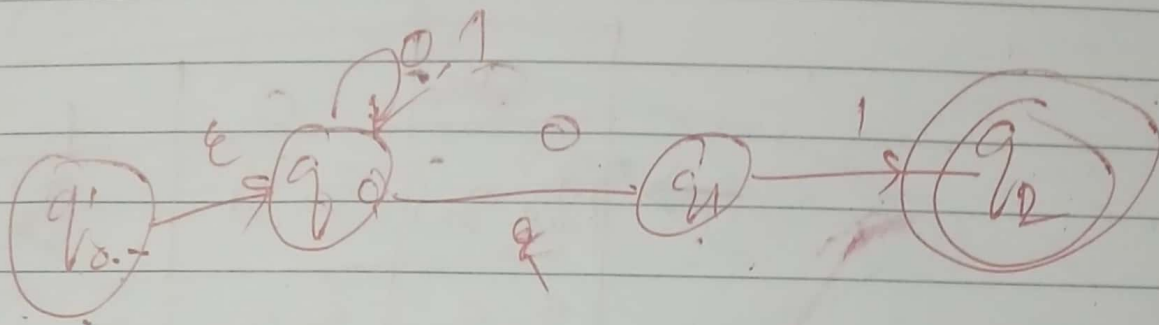
	ϵ	$+, -$	$\cdot$	$0, 1, \dots, 9$
q_0	$\{q_1\}$	$\{q_1\}$	ϕ	ϕ
q_1	ϕ	ϕ	$\{q_2\}$	$\{q_1, q_4\}$
q_2	ϕ	ϕ	ϕ	$\{q_3\}$
q_3	$\{q_5\}$	ϕ	ϕ	$\{q_3\}$
q_4	ϕ	ϕ	$\{q_3\}$	ϕ
q_5	ϕ	ϕ	ϕ	ϕ

$$\text{ECLOSE}(1) = \{1, 2, 3, 4, 6\}$$

$$(2) = \{2, 3, 6\}$$

Eliminating λ -Transitions.

$\Sigma = \{0, 1\}$ Date: _____

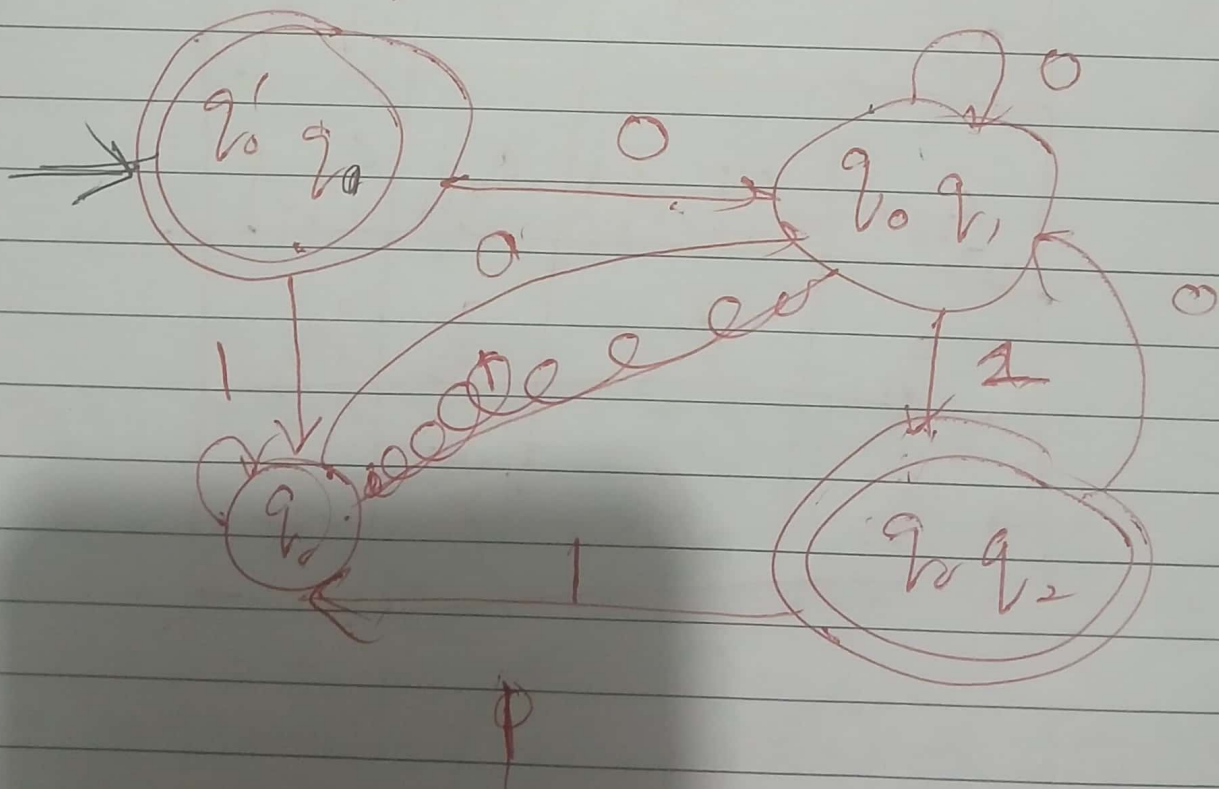


λ -NFA

State	0	1	ϵ
q_0'	$\emptyset$	$\emptyset$	$\{q_0', q_0\}$
q_0	$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0\}$
q_1	$\emptyset$	q_2	$\{q_1\}$
q_2	$\emptyset$	$\emptyset$	q_2

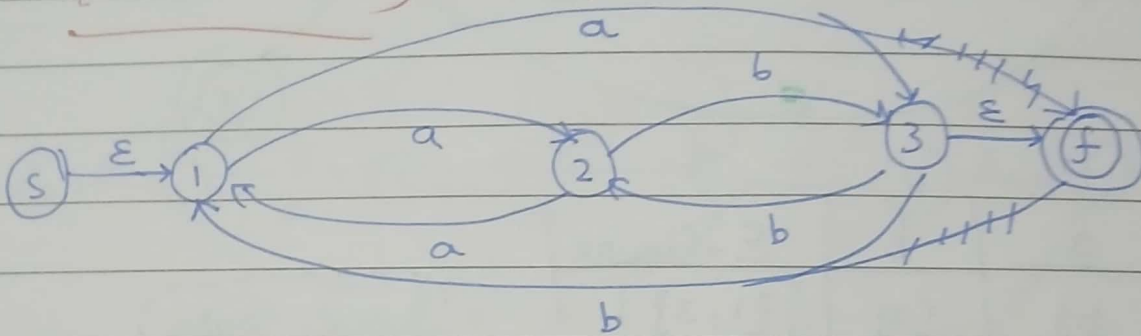
DFA

State	0	1
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_1	q_0
q_3	q_1	q_0



$$\emptyset \cup \{q_0, q_1\} = \{q_0, q_1\}$$

$$\emptyset \cup \{q_0\} = \{q_0\}$$



⇒ Example that we did in class:

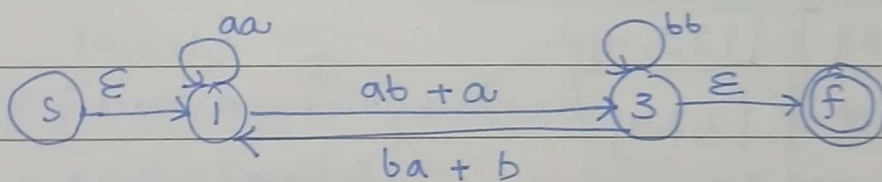
Eliminate (2):

$$1-2-1 \Rightarrow aa$$

$$1-2-3 \Rightarrow ab$$

$$3-2-1 \Rightarrow ba$$

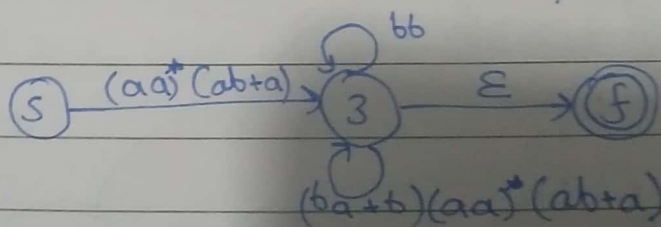
$$3-2-3 \Rightarrow bb$$



Eliminate (1):

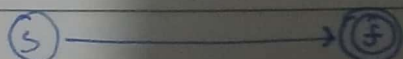
$$S \rightarrow 1 \rightarrow 3 \Rightarrow \epsilon(aa)^*(ab+a)$$

$$3 \rightarrow 1 \rightarrow 3 \Rightarrow (ba+b)(aa)^*(ab+a)$$



Eliminate (3):

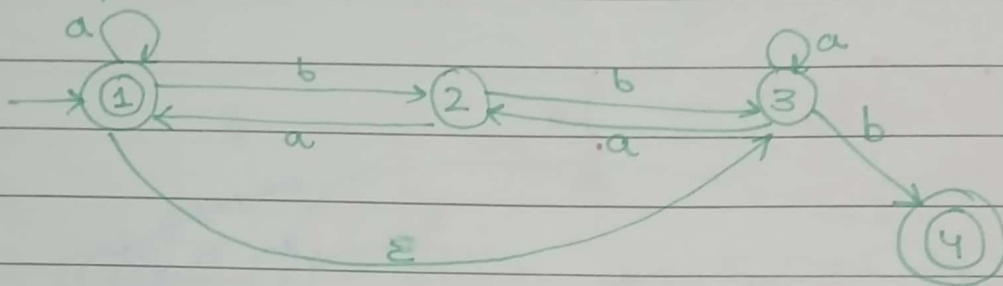
$$S \rightarrow 3 \rightarrow f \Rightarrow (aa)^*(ab+a)[(bb)^* + (ba+b)(aa)^*(ab+a)]^*$$



E-NFA $\rightarrow$ DFA

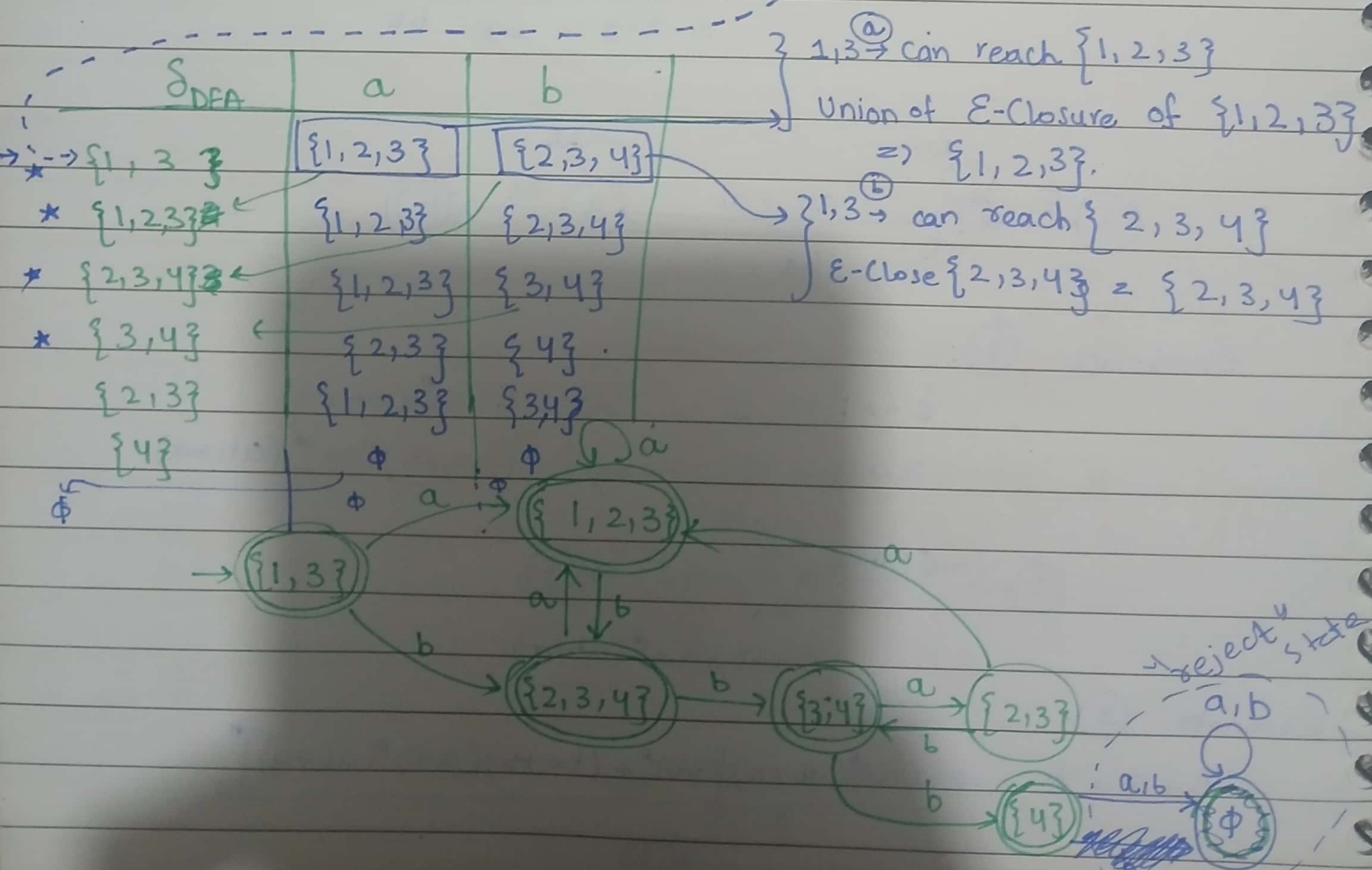
Date: _____

Another Example:



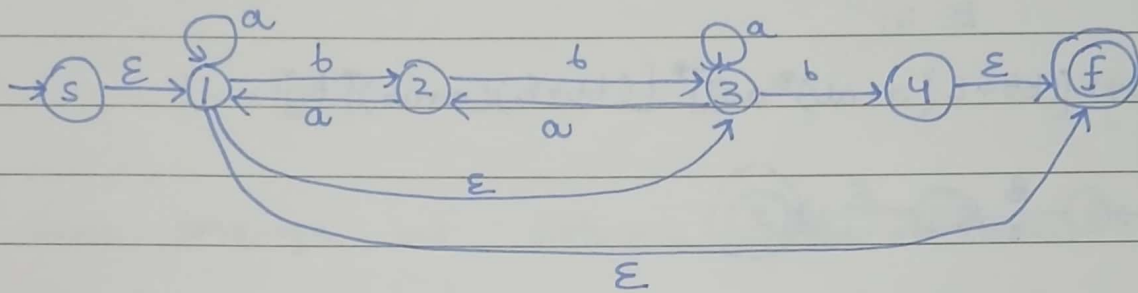
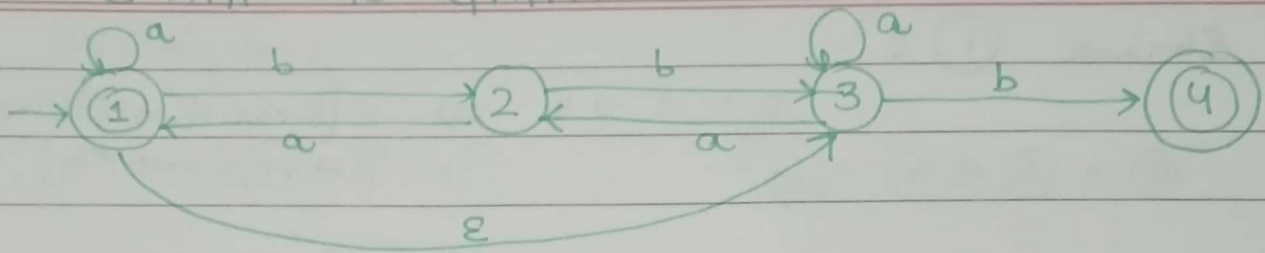
	a	b	ϵ -Closure
$\rightarrow$ * 1	{1}	{2}	{1, 3}
2	{1}	{3}	{2}
3	{2, 3}	{4}	{3}
* 4	ϕ	ϕ	{4}

Initial State



ϵ -NFA to GNFA:

Date: _____



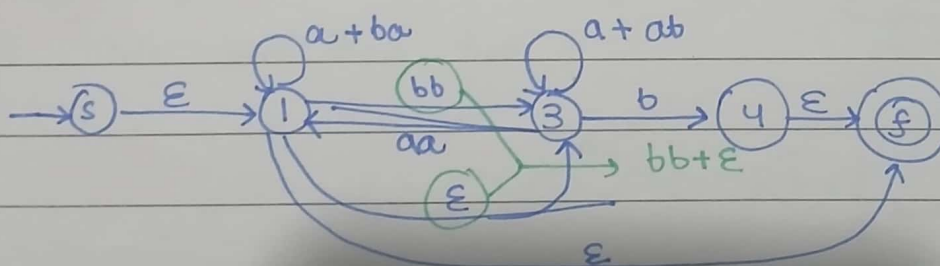
Eliminate (2):

$$1-2-1 \Rightarrow ba$$

$$1-2-3 \Rightarrow bb$$

$$3-2-1 \Rightarrow aa$$

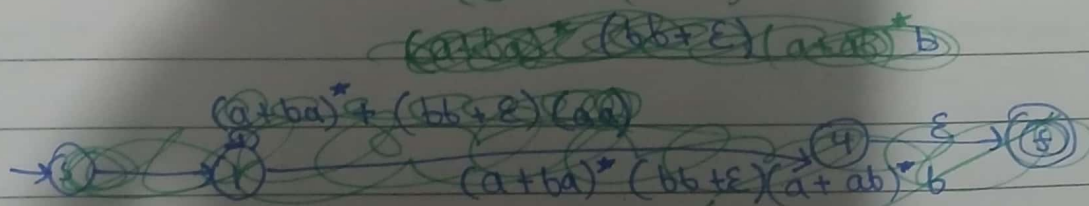
$$3-2-3 \Rightarrow ab$$



Eliminate (3):

$$1-3-1 \Rightarrow (a+ba)^*(bb+\epsilon)(a+ab)^*aa$$

$$1-3-4 \Rightarrow (a+ba)^*(bb+\epsilon)(a+ab)^*b$$



Structure of DFA.

- Language $\Rightarrow$ problem statement $\Rightarrow$ EVEN-EVEN?

$\Sigma = \{0, 1\}$ alphabet

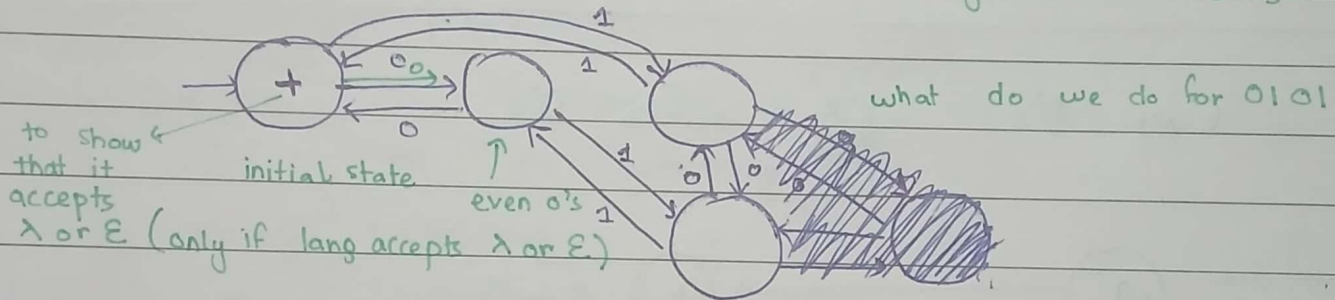
$L = \{ \epsilon, 00, 11, 1010, 0011, 1001, 0110, \dots \}$

everything is occurring even number of times.

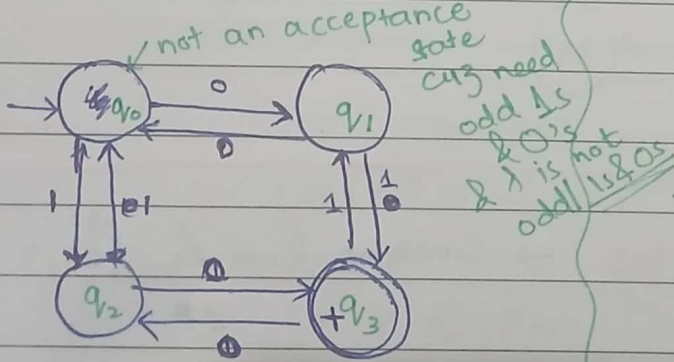
Machines:

need: $\rightarrow$ alphabet, states, transitions, start initial state, final state

start state will accept λ or ϵ always, in case language contains?



odd — odd



odd 1's & even 0's

even 1's & odd 0's

odd 1's | even 1's | odd 0's | even 0's

Transition table for odd-odd:

	0	1
q_0	q_1	q_2
q_1	q_0	q_3
q_2	q_3	q_0
q_3	q_2	q_1

even-even

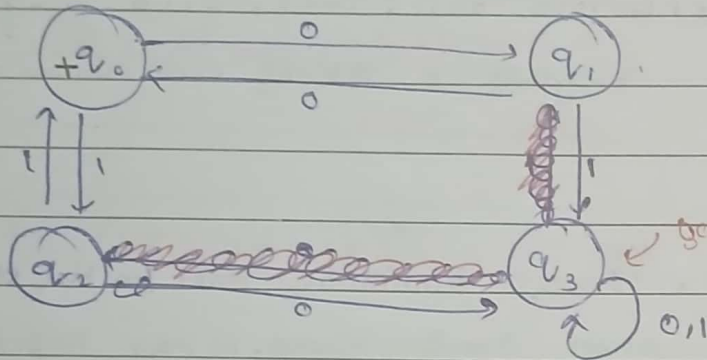
odd 0's, even 1's

odd 1's, even 0's

odd odd

Even Chunks of 0's & 1's

00 11 00
00 00 11



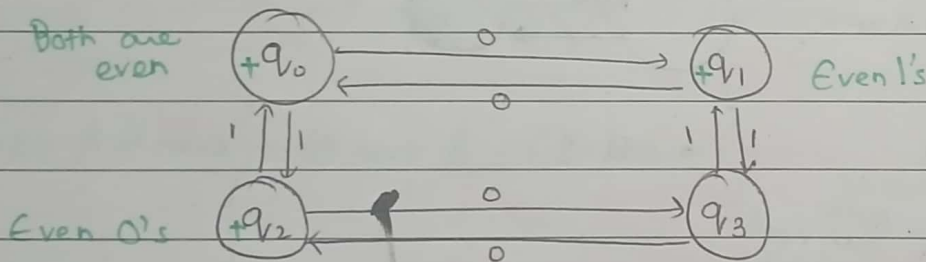
⇒ T Table

	0	1
q_0	q_1	q_2
q_1	q_0	q_3
q_2	q_3	q_0
q_3	q_3	q_3

Even 1's

OR

Even 0's



Regular Expression → RE → Something from Σ^* (eg: {0,1})

$\gamma^+ = \gamma \cdot \gamma^*$

$\gamma = (\gamma_1 + \gamma_2) \cdot (\gamma_3 + \gamma_4)$

$\gamma = (\gamma_1 \gamma_2) + \gamma_3$

→ $\gamma_1 + \gamma_2$ } union of 2 regular expressions
→ $\gamma_1 \gamma_2$ } concatenation
→ within (γ)

→ can be $\emptyset \Rightarrow L = \{\}$ $\neq \epsilon$

$L(\epsilon) \Rightarrow L = \{\epsilon\} \neq \emptyset$

a) to calculate the intersection, we will first need to generate the language then ^{find out} calculate the intersection

Date: _____

$$\Sigma = \{0, 1\}$$

• $\Sigma^* \rightarrow (0+1)^*$ ← all possible things.

• Even ~~more~~ Chunks.

is also valid ✓

$$\cancel{((aa)^*(bb)^*)^*} \quad (aa+bb)^* \\ (00+11)^*$$

Equivalent RE??

(Week 02, Lect # 2) Date: 29th January 2024

Languages

$\Sigma = \{0, 1\}$ for all these questions

- ① with 01 as substring.
- ② strings starting with 11
- ③ ending with 11
- ④ All strings starting & ending with same symbol.

R.E. for above languages:

all possible strings $\rightarrow (0+1)^*$

① $(0+1)^* 01 (0+1)^*$

② $11 \cdot (0+1)^*$

③ $(0+1)^* 11$

④ $\epsilon + 0(1+0)^*0 + 1(1+0)^*1 + 1+0$
 $L_4 = \epsilon, 11, 00, 1, 0,$

Mapping sets to RE.

$$\{0, 1\} \Rightarrow 0+1$$

$$\{0, 1\}^* \Rightarrow (0+1)^*$$

$$\{00+11\} \Rightarrow 00+11$$

starts & ends with ϵ

starts & ends on 1/0

we're missing these...

int 13-08-24
Date: _____

$$\Sigma = \{0, 1\}$$

- $\Sigma^* \rightarrow (0+1)^*$ ← all possible things.
- Even ~~to~~ Chunks.

✓ is also valid ✓

$$(\cancel{(aa)^*} \cancel{(bb)^*})^* \quad (aa+bb)^* \\ (00+11)^*$$

Equivalent RE??

(Week 02, Lect # 2) DATE: 29th January 2024

Languages

$\Sigma = \{0, 1\}$ for all these questions

- ① with 01 as substring.
- ② strings starting with 11
- ③ " ending with 11
- ④ All strings starting & ending with same symbol.

R.E. for above languages:

all possible strings $\rightarrow (0+1)^*$

① $(0+1)^* 01 (0+1)^*$

② $11(0+1)^*$

③ $(0+1)^* 11$

④ $\epsilon + 0(1+0)^* 0 + 1(1+0)^* 1 + 1+0$

$L_4 = \epsilon, 11, 00, 110, 101, \dots$

we're missing these...

Mapping sets to RE.

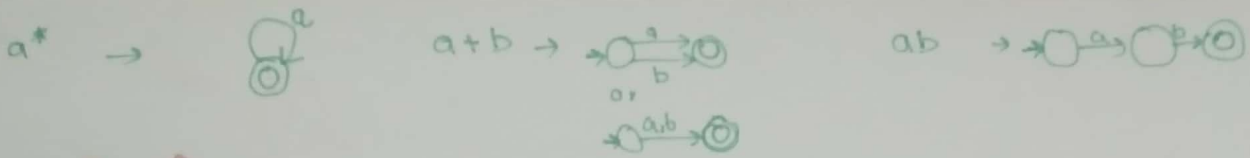
$$\{0, 1\} \Rightarrow 0+1$$

$$\{0, 1\}^* \Rightarrow (0+1)^*$$

$$\{00+11\} \Rightarrow 00+11$$

starts & ends with ϵ

starts & ends on 1/0



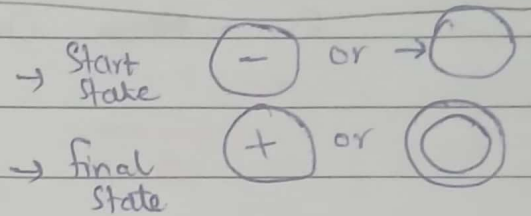
Self Study

Date: _____

Finite State machine $\rightarrow$ FSM

Finite Automata $\rightarrow$ FA (finite Acceptor)

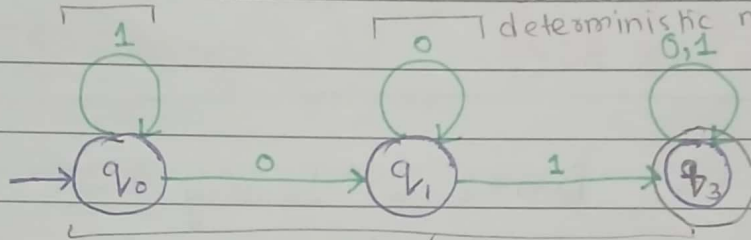
DFA's for examples on previous pages:



①

Essential States go first.

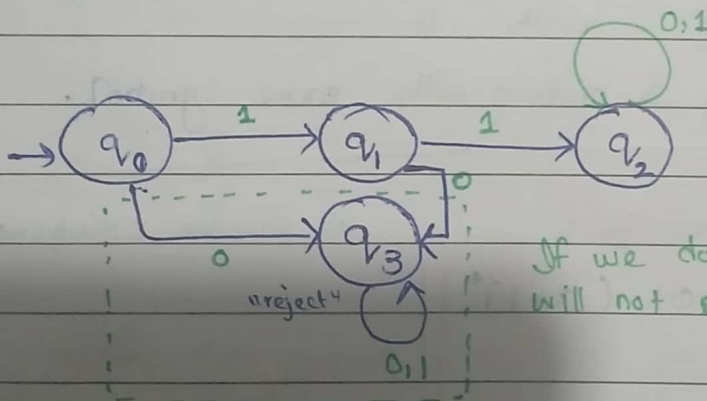
many times before 0 one can occur however putting this 0 here instead of q_0 so that it doesn't violate deterministic nature of DFA



$\frac{10001001}{q_0 q_1 q_3 q_3}$

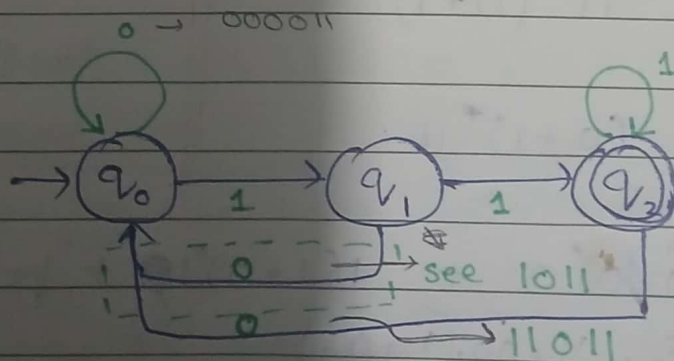
once this occurs, then 0 & 1 can occur however many times.

②



If we don't add this then DFA will not be deterministic.

③



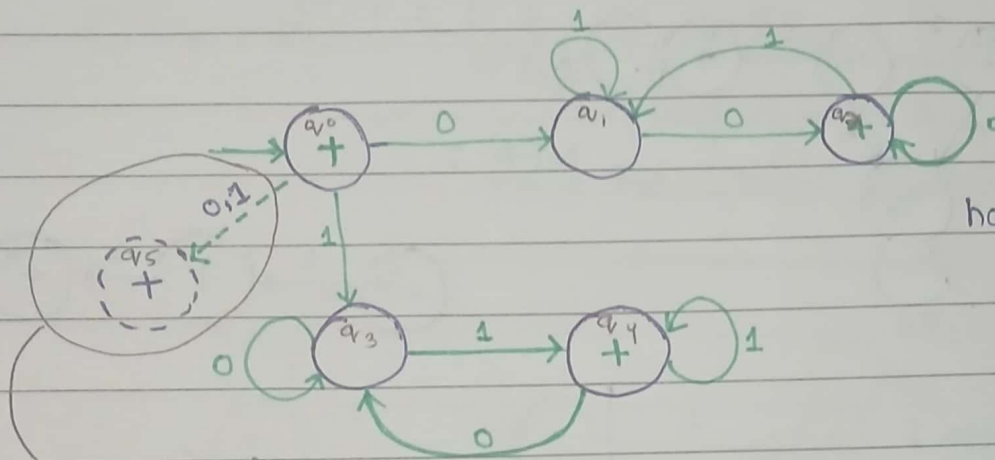
$\delta(Q \times \Sigma) \rightarrow Q \leadsto$ Deterministic Transition function

$\delta(Q \times \Sigma) \rightarrow \text{Subset of } Q (\text{including } \emptyset) \leadsto$ NonDeterministic Transition function

Date: _____

(4)

$$\epsilon + 0 + 1 + 0(1+0)^*0 + 1(1+0)^*1$$



how to accept 0 & 1 here?

→ makes it non deterministic

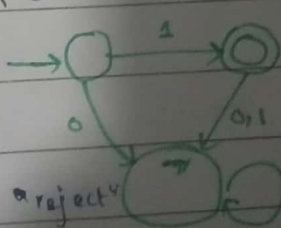
⇒ In deterministic, you can land on only one state.

⇒ TRANSITION TABLE :

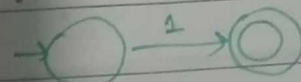
DNFA	0	1
→ * q_0	(q_1, q_5)	(q_3, q_5)
q_1	q_2	q_1
* q_2	q_2	q_1
q_3	q_3	q_4
* q_4	q_3	q_4
* q_5	$\emptyset$	$\emptyset$

Language is let's say "1" only.

DFA :

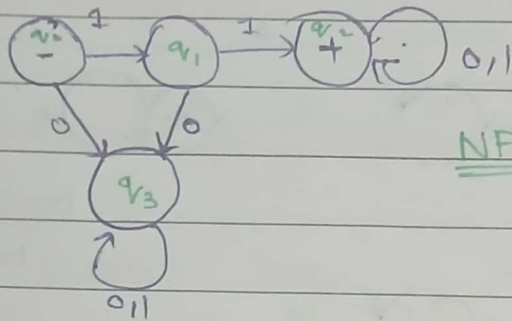


NFA :

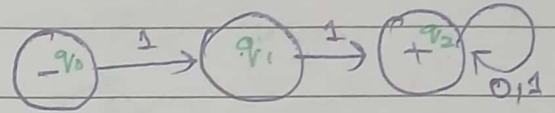


for every state we need to say what happens when we get a character.

DFA for language starting with "11"



NFA :



⇒ Transition Table :

DFA

Transition function

δ_{DFA}	0	1
q_0	q_3	q_1
q_1	q_3	q_2
q_2	q_2	q_2
q_3	q_3	q_3

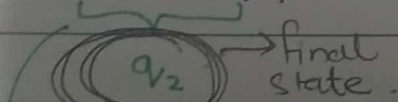
NFA

δ_{NFA}	0	1
q_0	ϕ	q_1
q_1	ϕ	q_2
q_2	q_2	q_2

$$\delta(q_0, 110) \Rightarrow \delta(q_0, \delta(q_1, 10))$$

$$\Rightarrow \delta(q_0, \delta(q_1, \delta(q_2, 0)))$$

if we reduce more, it'll be λ or ϵ



if we split more:
 $\delta(q_2, \delta(q_2, \lambda))$

$$\delta(q_n, \epsilon) \Rightarrow q_n$$

shows no movement

σ → sigma or δ

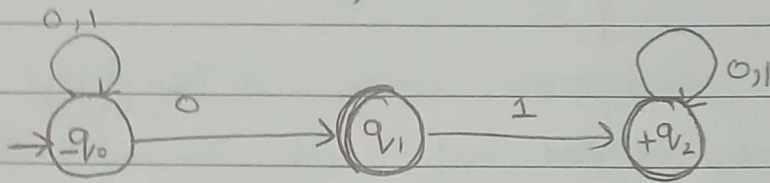
$\delta \rightarrow$ delta
(transition).

NFA → DFA

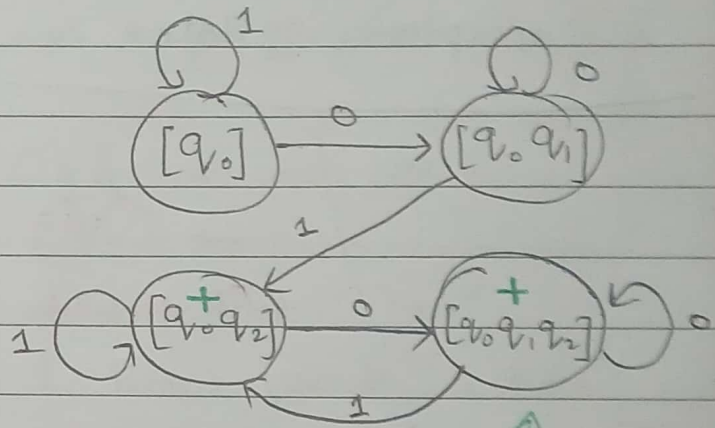
subset of all states reached by a string Date: _____

$$\hat{\delta}(q_0, \Sigma^*) \rightarrow \{q_0 \cup \dots \cup q_t\}$$

↳ If any one of these belong to final state then string is accepted.



δ_{NFA}	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\{q_2\}$	$\{q_2\}$



Convert NFA to DFA ??

→ first separate out subsets ←

$\emptyset$	$\emptyset$	$\emptyset$	] not needed
$\{q_0\}$	$\{[q_0, q_1]\}$	$[q_0]$	
$\{q_1\}$	$[\emptyset]$	$[q_2]$	] not needed
$\{q_2\}$	$[q_2]$	$[q_2]$	
$\{q_0, q_1\}$	$q_0, q_1 \cup \emptyset$	$q_0 \cup q_2$	
$\{q_0, q_2\}$	$[q_0, q_1, q_2]$	$[q_0, q_2]$	
$\{q_1, q_2\}$	$[q_2]$	$[q_2]$	] not needed
$\{q_0, q_1, q_2\}$	$[q_0, q_1, q_2]$	$[q_0, q_2]$	

using this table we can make the DFA

	0	1
q_0	$q_0 q_1$	q_0
$q_0 q_1$	$q_0 q_1$	$q_0 q_2$
* $q_0 q_2$	$q_0 q_1 q_2$	$q_0 q_2$
* $q_0 q_1 q_2$	$q_0 q_1 q_2$	$q_0 q_2$

Easier way to do the thing on the previous page!

- q_2 was the acceptance state in the NFA
- So any state with q_2 will be considered as an acceptance state.

→ Quiz SOLUTION ←

((Rough work on OG copy))

- ① $b^* a^*$
- ② $a^* + (b^* a^* b b^*)^*$
- ③ $a^* + b^*$
- ④ ~~ab~~ $(a+b)^* \overset{ba}{ab} (a+b)^*$
- ⑤ $(a+b)(a+b)(a+b)^* + \lambda$
- ⑥ $\lambda + b^* + ab^* + ba^* + b^* ab^* ab^* ab^* (a+b)^*$
- ⑦ $(ab+ba)[aa+bb+(ab+ba)(aa+bb)^*(ba+ab)]^*$
- ⑧ $b^* a (b^* ab^* a)^* b^* b$
- ⑨ $(b^* ab b^* abb^*)^*$

Questions for Assignment

Date: _____

→ Regular Expressions

→ NFA

→ DFA

} ((scenario based } direct))

• Regular expression where:

→ w is any string except 11 & 111

→ w where every odd position in w is 1

→ w starts with 0 & has odd length or starts with 1 & has even length

→ contains at least ~~2~~ two 0 's & at most one 1 .

→ DFA for a language which ensures that " aa " occurs min 0 times & max 1 time. (exactly 1 time).

→ A DFA for ~~accept~~ a fridge door. {open, close, wait}

States → locked | unlocked | open | closed

